HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1




# BÁO CÁO ASSIGNMENT 05

# HỌC PHẦN: PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH


| | |
|---|---|
| **Giảng viên** | **: PGS.TS. Trần Đình Quế** |
| **Nhóm học phần** | **: 05** |
| **Họ và tên** | **: Nguyễn Đức Trung** |
| **Mã sinh viên** | **: B23DCCN857** |
| **Lớp** | **: D23CNPM01** |


**HÀ NỘI 2026**

# MỤC LỤC

**DANH MỤC HÌNH ẢNH ..... 7**

**1. GIỚI THIỆU VÀ MỤC TIÊU CỦA ASSIGNMENT 05 ..... 9**

**1.1. Bối cảnh ..... 9**

**1.2. Mục tiêu ..... 9**

**1.3. Phạm vi và cấu trúc thực nghiệm ..... 10**

**1.4. Ý nghĩa của cách tiếp cận ..... 10**

**2. CƠ SỞ LÝ THUYẾT VỀ MẠNG NƠ-RON VÀ CNN ..... 11**

**2.1. Hàm toán học và mô hình học ..... 11**

**2.2. Neuron nhân tạo ..... 11**

**2.3. Neuron dưới góc nhìn PyTorch ..... 12**

**2.4. Hàm kích hoạt và tính phi tuyến ..... 12**

**2.5. Tích chập (Convolution) ..... 13**

**2.6. Local connectivity và weight sharing ..... 13**

**2.7. Padding, stride và kích thước feature map ..... 14**

**2.8. Pooling ..... 14**

**2.9. Flatten và Fully Connected Layer ..... 14**

**2.10. CNN như một hàm hợp ..... 15**

**2.11. Lan truyền thuận, hàm mất mát và tối ưu ..... 15**

**2.12. Train, validation và test ..... 16**

**3. SỰ PHÁT TRIỂN CỦA CÁC KIẾN TRÚC CNN ..... 16**

**3.1. Từ Basic CNN đến các kiến trúc sâu hơn ..... 16**

**3.2. LeNet** ..... **17**

**3.3. AlexNet** ..... **18**

**3.4. VGG** ..... **18**

**3.5. Inception, ResNet, DenseNet và các hướng phát triển khác** ..... **18**

**4. CƠ SỞ LÝ THUYẾT VỀ CÁC KỸ THUẬT CẢI TIẾN CNN** ..... **19**

**4.1. Batch Normalization** ..... **19**

**4.2. Residual Connection** ..... **19**

**4.3. Ý nghĩa của identity path** ..... **20**

**4.4. Attention và Squeeze-and-Excitation** ..... **20**

**4.5. Attention không đồng nghĩa với luôn tốt hơn** ..... **21**

**4.6. Conv2d và Conv1d** ..... **21**

**4.7. Standardization và tránh rò rỉ dữ liệu** ..... **21**

**5. DỮ LIỆU VÀ BÀI TOÁN THỰC NGHIỆM** ..... **22**

**5.1. MNIST** ..... **22**

**5.2. Dạng tensor và DataLoader** ..... **22**

**5.3. Fashion-MNIST** ..... **23**

**5.4. Ý nghĩa của Fashion-MNIST trong thực nghiệm** ..... **25**

**5.5. Diabetes dataset** ..... **25**

**5.6. Xử lý giá trị 0** ..... **26**

**5.7. Chia dữ liệu Diabetes** ..... **27**

**5.8. So sánh đặc điểm ba dataset** ..... **28**

**6. THIẾT KẾ VÀ CÀI ĐẶT CÁC MÔ HÌNH CNN** ..... **29**

**6.1. Nhóm M1–M4** ..... **29**

**6.2. M1 Basic CNN** ..... **29**

**6.3. M2 CNN với Batch Normalization ..... 30**

**6.4. M3 Residual CNN ..... 32**

**6.5. M4 Attention CNN ..... 34**

**6.6. Mô hình CNN cho Diabetes ..... 36**

**6.7. Classical CNN – LeNet, AlexNet và VGG ..... 42**

**6.8. Kiểm tra output shape và parameter count ..... 42**

**6.9. Tổng hợp kiến trúc cuối cùng ..... 42**

**7. QUY TRÌNH HUẤN LUYỆN VÀ ĐÁNH GIÁ ..... 43**

**7.1. Thiết lập huấn luyện ..... 43**

**7.2. Lựa chọn best state ..... 43**

**7.3. Đánh giá bằng test set ..... 44**

**7.4. Metrics ..... 44**

**8. KẾT QUẢ THỰC NGHIỆM ..... 44**

**8.1. Kết quả MNIST ..... 44**

**8.2. Kết quả Fashion-MNIST ..... 45**

**8.3. Kết quả Diabetes ..... 46**

**8.4. Classification metrics ..... 47**

**8.5. Accuracy improvement so với Basic CNN ..... 47**

**8.6. Complexity ..... 48**

**9. TRỰC QUAN HÓA VÀ PHÂN TÍCH BIỂU ĐỒ ..... 48**

**9.1. Vai trò của visualization ..... 48**

**9.2. Training và validation loss – MNIST ..... 49**

**9.3. Training và validation loss – Fashion-MNIST ..... 50**

**9.4. Training và validation loss – Diabetes ..... 51**

**9.5. Test accuracy – MNIST ..... 52**

**9.6. Test accuracy – Fashion-MNIST ..... 53**

**9.7. Test accuracy – Diabetes ..... 54**

**9.8. Training time ..... 55**

**9.9. Trainable parameters ..... 56**

**9.10. Confusion Matrix – MNIST ..... 57**

**9.11. Confusion Matrix – Fashion-MNIST ..... 58**

**9.12. Confusion Matrix – Diabetes ..... 59**

**10. SO SÁNH, THẢO LUẬN VÀ NHẬN XÉT ..... 59**

**10.1. So sánh M1–M4 trên MNIST ..... 59**

**10.2. So sánh M1–M4 trên Fashion-MNIST ..... 60**

**10.3. So sánh M1–M4 trên Diabetes ..... 60**

**10.4. Classical CNN và model cải tiến ..... 61**

**10.5. Accuracy và test loss không phải cùng một khái niệm ..... 61**

**10.6. Accuracy, Precision, Recall và F1 ..... 61**

**10.7. Complexity và hiệu quả ..... 62**

**10.8. Vai trò của reproducibility ..... 62**

**10.9. Ý nghĩa học thuật của Assignment 05 ..... 62**

**11. HẠN CHẾ CỦA THỰC NGHIỆM ..... 63**

**11.1. Số epoch còn hạn chế ..... 63**

**11.2. Dataset Diabetes nhỏ ..... 63**

**11.3. Conv1d trên dữ liệu bảng ..... 63**

**11.4. Classical architectures là phiên bản thích nghi ..... 63**

**11.5. Training time phụ thuộc môi trường ..... 64**

**11.6. Không có tuning sâu ..... 64**

**11.7. Hướng cải thiện ..... 64**

**12. KẾT LUẬN ..... 64**

**PHỤ LỤC: LINK DATASET VÀ SOURCE CODE ..... 66**

# DANH MỤC HÌNH ẢNH

Hình 1. Quy trình thực nghiệm tổng quát của Assignment 05. ..... 10
Hình 2. CNN được nhìn như một hàm hợp của nhiều phép biến đổi. ..... 15
Hình 3. Một số mẫu ảnh MNIST được trực quan hóa trong quá trình chuẩn bị dữ liệu. ..... 22
Hình 4. DataLoader của MNIST ..... 23
Hình 5. Một số mẫu Fashion-MNIST được trực quan hóa. ..... 24
Hình 6. Training/Validation/Test samples của Fashion-Mnist ..... 24
Hình 8. Xử lý giá trị 0 trong Diabete ..... 27
Hình 9. Kết quả Dataset summary trong notebook ..... 28
Hình 10. Định nghĩa hàm cho M1 ..... 30
Hình 11. Định nghĩa hàm cho M2 ..... 31
Hình 12. So sánh tham số giữa M1 và M2 ..... 31
Hình 13. Định nghĩa ResidualBlock ..... 32
Hình 14. Định nghĩa ResidualCNN ..... 33
Hình 15. Định nghĩa SEBlock ..... 34
Hình 16. Định nghĩa AttentionCNN ..... 35
Hình 17. Định nghĩa DiabetesBasicCNN ..... 36
Hình 18. Định nghĩa hàm BatchNorm cho Diabete ..... 37
Hình 19. Định nghĩa ResidualBlock1D ..... 38
Hình 20. Định nghĩa DiabetesResidualCNN ..... 39
Hình 21. Định nghĩa SEBlock1D cho Diabetes ..... 40
Hình 22. Định nghĩa DiabetesAttentionCNN ..... 41
Hình 23. Đường loss huấn luyện và validation của bảy mô hình trên MNIST. ..... 49

Hình 24. Đường loss huấn luyện và validation của các mô hình trên Fashion-MNIST. ...50
Hình 25. Đường loss huấn luyện và validation của bốn mô hình Diabetes. .....51
Hình 26. So sánh test accuracy của bảy mô hình trên MNIST. .....52
Hình 27. So sánh test accuracy trên Fashion-MNIST. .....53
Hình 28. So sánh test accuracy của bốn mô hình Diabetes. .....54
Hình 29. Thời gian huấn luyện các mô hình ảnh trên MNIST và Fashion-MNIST. .....55
Hình 30. So sánh số tham số trainable của bảy mô hình ảnh. .....56
Hình 31. Confusion Matrix của M7 Adapted VGG trên MNIST. .....57
Hình 32. Confusion Matrix của M6 Adapted AlexNet trên Fashion-MNIST. .....58
Hình 33. Confusion Matrix của M3 Residual trên Diabetes. .....59

# 1. GIỚI THIỆU VÀ MỤC TIÊU CỦA ASSIGNMENT 05

## 1.1. Bối cảnh

Convolutional Neural Network (CNN) là một nhóm kiến trúc học sâu đặc biệt phù hợp với dữ liệu có cấu trúc không gian hoặc cấu trúc cục bộ. Trong xử lý ảnh, CNN có khả năng học các đặc trưng từ mức thấp như cạnh, đường biên và họa tiết đến các đặc trưng phức tạp hơn thông qua nhiều lớp tích chập. Assignment 05 đặt trọng tâm không chỉ vào việc chạy một mô hình CNN có sẵn mà còn vào việc hiểu CNN dưới góc nhìn hàm toán học, cách các phép toán cơ bản được cài đặt bằng Python/PyTorch, quá trình tiến hóa kiến trúc và ảnh hưởng của các cơ chế cải tiến.

Notebook đã triển khai đúng hướng tiếp cận này. Phần đầu giải thích hàm toán học, neuron nhân tạo, ReLU, convolution, pooling và cách ghép chúng thành một CNN. Phần tiếp theo xây dựng các mô hình từ cơ bản đến cải tiến, sau đó mở rộng sang các kiến trúc kinh điển LeNet, AlexNet và VGG dưới dạng thích nghi cho ảnh 28×28.

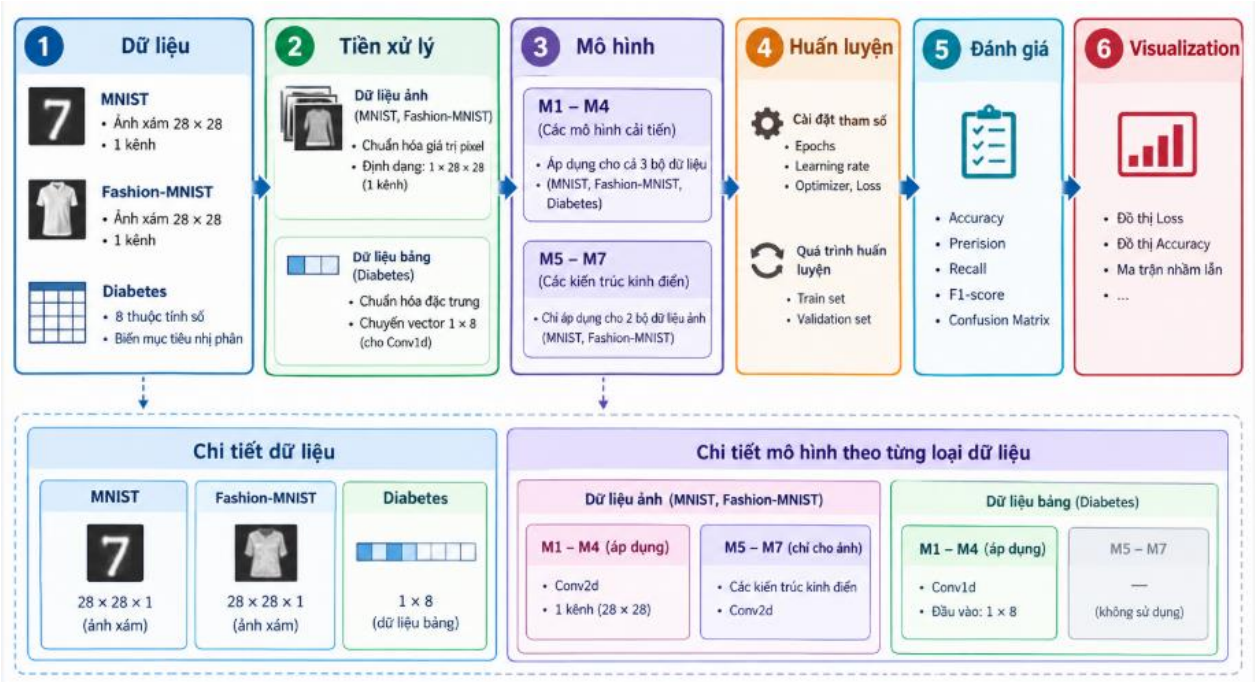
## 1.2. Mục tiêu

- Hiểu CNN dưới góc nhìn của các hàm toán học và hàm hợp.
- Biểu diễn các phép toán cơ bản bằng công thức và mã Python/PyTorch.
- Hiểu sự phát triển của các kiến trúc CNN và vấn đề mà từng cải tiến hướng tới.
- Xây dựng một CNN cơ bản và ba CNN cải tiến: Batch Normalization, Residual và Attention.
- Áp dụng CNN cho MNIST, Fashion-MNIST và Diabetes.
- Đánh giá mô hình bằng Accuracy, Precision, Recall, F1-score, Test Loss và Confusion Matrix.
- So sánh độ chính xác với số tham số và thời gian huấn luyện.
- Trực quan hóa quá trình hội tụ và kết quả dự đoán.

## 1.3. Phạm vi và cấu trúc thực nghiệm

Thực nghiệm sử dụng ba bộ dữ liệu. MNIST và Fashion-MNIST là dữ liệu ảnh xám có kích thước 28×28 nên được biểu diễn theo dạng một kênh và xử lý bằng Conv2d. Diabetes là dữ liệu dạng bảng gồm tám thuộc tính số và một biến mục tiêu nhị phân; notebook chuyển vector đặc trưng thành chuỗi một chiều có dạng 1×8 để minh họa việc sử dụng Conv1d.

Bốn mô hình cải tiến M1–M4 được dùng trên cả ba bộ dữ liệu. Ba kiến trúc kinh điển M5–M7 chỉ được dùng cho hai bộ dữ liệu ảnh. Cách tổ chức này vừa đáp ứng mục tiêu nghiên cứu các cải tiến CNN, vừa cho phép minh họa sự khác biệt giữa kiến trúc dành cho dữ liệu ảnh và cách thích nghi convolution cho dữ liệu đặc trưng một chiều.




*Hình 1. Quy trình thực nghiệm tổng quát của Assignment 05.*

## 1.4. Ý nghĩa của cách tiếp cận

Điểm quan trọng của bài thực hành là không xem CNN như một hộp đen. Từ một neuron đơn giản, người học có thể quan sát cách các phép toán tuyến tính, hàm phi tuyến, tích chập và pooling được ghép thành một hàm phức tạp. Sau đó, Batch Normalization,

residual connection và attention được thêm vào để thay đổi cách thông tin được chuẩn hóa, truyền qua mạng và được nhấn mạnh.

# 2. CƠ SỞ LÝ THUYẾT VỀ MẠNG NƠ-RON VÀ CNN

## 2.1. Hàm toán học và mô hình học

Notebook bắt đầu từ cách nhìn một mô hình học máy như một hàm tham số hóa. Với hàm tổng quát y = f(x), mạng nơ-ron học một họ hàm có tham số θ, ký hiệu $y = f_\theta(x)$. Dữ liệu huấn luyện cung cấp các cặp đầu vào và mục tiêu để thuật toán tối ưu tìm ra bộ tham số phù hợp.

Ý tưởng này rất quan trọng khi chuyển sang CNN. Một CNN không phải một phép toán duy nhất mà là một chuỗi các phép biến đổi. Mỗi lớp nhận đầu ra của lớp trước, thực hiện một hàm có tham số hoặc một phép biến đổi cố định, sau đó truyền kết quả sang lớp tiếp theo.

$$y = f_\theta(x)$$

Trong quá trình học, θ được điều chỉnh nhằm làm giảm một hàm mất mát. Với bài toán phân loại, đầu ra của mô hình được so sánh với nhãn thật thông qua CrossEntropyLoss đối với bài toán nhiều lớp hoặc BCEWithLogitsLoss đối với bài toán nhị phân.

## 2.2. Neuron nhân tạo

Neuron nhân tạo là đơn vị cơ bản của mạng nơ-ron. Notebook minh họa neuron bằng tích vô hướng giữa vector đầu vào và vector trọng số, cộng thêm bias. Đây là một ví dụ trực tiếp để liên hệ công thức toán học với lớp nn.Linear trong PyTorch.

$$z = \Sigma(w_i x_i) + b = w^T x + b$$

Trong đó x là vector đầu vào, w là vector trọng số, b là bias và z là giá trị trước activation. Trọng số biểu diễn mức độ đóng góp của từng đầu vào. Bias cho phép hàm quyết định dịch chuyển theo trục đầu ra.

## 2.3. Neuron dưới góc nhìn PyTorch

Notebook sử dụng một vector x = [2, 3, 1], trọng số w = [0.5, −1.0, 2.0] và b = 0.5. Sau đó cùng phép tính được tái hiện bằng nn.Linear(3, 1). Đây là một ví dụ có giá trị vì nó cho thấy một lớp học sâu thực chất vẫn thực hiện phép toán tuyến tính quen thuộc.

```
x = torch.tensor([[2.0, 3.0, 1.0]])
neuron = nn.Linear(3, 1)

with torch.no_grad():
    neuron.weight[:] = torch.tensor([[0.5, -1.0, 2.0]])
    neuron.bias[:] = torch.tensor([0.5])

y = neuron(x)
```

Việc kiểm tra đầu ra giữa phép tính NumPy và PyTorch giúp xác nhận sự tương ứng giữa công thức và cài đặt.

## 2.4. Hàm kích hoạt và tính phi tuyến

Nếu một mạng chỉ gồm các phép biến đổi tuyến tính thì nhiều lớp tuyến tính ghép lại vẫn tương đương với một phép biến đổi tuyến tính. Vì vậy, mạng sẽ không có đủ khả năng biểu diễn các quan hệ phi tuyến phức tạp. Hàm kích hoạt được đưa vào để phá vỡ tính tuyến tính này.

$$h = \sigma(z)$$

Notebook sử dụng ReLU làm ví dụ chính. ReLU giữ nguyên giá trị dương và đưa giá trị âm về 0. Đây là hàm đơn giản, tính toán nhanh và thường được sử dụng trong CNN.

$$ReLU(x) = max(0, x)$$

## 2.5. Tích chập (Convolution)

Convolution là phép toán cốt lõi của CNN. Thay vì nối toàn bộ đầu vào với toàn bộ neuron như fully connected layer, convolution quan sát các vùng cục bộ và áp dụng cùng một kernel tại nhiều vị trí. Điều này tạo ra hai đặc tính quan trọng: local connectivity và weight sharing.

$$y_i = \Sigma(j=0 \rightarrow k-1)\ x_{i+j} w_j + b$$

Trong notebook, một vector một chiều được dùng để minh họa trực tiếp. Kernel [0.5, −1.0, 0.5] lần lượt trượt trên các cửa sổ của dữ liệu. Ở mỗi vị trí, các phần tử tương ứng được nhân với nhau rồi cộng lại.

```
x = np.array([2, 1, 3, 4, 2])
kernel = np.array([0.5, -1.0, 0.5])


for i in range(len(x) - len(kernel) + 1):
    window = x[i:i + len(kernel)]
    value = np.sum(window * kernel)
```

Trong ảnh hai chiều, nguyên lý tương tự được mở rộng theo cả chiều cao và chiều rộng. Một Conv2d có thể học nhiều kernel, mỗi kernel tạo ra một feature map. Những feature map này là biểu diễn mới của ảnh, trong đó các mẫu mà kernel học được sẽ tạo ra đáp ứng mạnh hơn.

## 2.6. Local connectivity và weight sharing

Local connectivity giúp CNN khai thác cấu trúc lân cận. Một pixel thường có quan hệ mạnh hơn với các pixel gần nó so với một pixel rất xa. Weight sharing giúp cùng một bộ trọng số phát hiện một mẫu tại nhiều vị trí khác nhau, làm giảm số tham số so với kết nối đầy đủ.

## 2.7. Padding, stride và kích thước feature map

Mặc dù notebook không xây dựng một cell riêng cho từng công thức kích thước, các mô hình sử dụng kernel 3×3 với padding=1 và pooling kích thước 2. Đây là những tham số quan trọng để kiểm soát kích thước không gian.

$$H_out = floor((H_in + 2P - K)/S) + 1$$

$$W_out = floor((W_in + 2P - K)/S) + 1$$

Với H_in = W_in = 28, K = 3, P = 1 và S = 1, kích thước sau convolution vẫn là 28×28. Sau MaxPool2d(2), kích thước giảm còn 14×14; pooling lần thứ hai đưa kích thước về 7×7. Vì vậy, lớp phân loại của Basic CNN nhận 32×7×7 giá trị trước khi Flatten.

Cách tính kích thước này cũng giải thích tại sao việc thiết kế classifier phải khớp với output của phần feature extractor. Nếu kích thước không được tính đúng, lớp Linear sẽ không nhận đúng số chiều đầu vào.

## 2.8. Pooling

Pooling là một phép downsampling. Max pooling chọn giá trị lớn nhất trong mỗi cửa sổ. Notebook minh họa [1, 5, 2, 4] với cửa sổ 2 để tạo ra [5, 4]. Trong CNN, pooling làm giảm độ phân giải không gian, giảm chi phí tính toán và giữ lại các đáp ứng nổi bật.

$$y_i = max(x_i, \ldots, x_{i+k-1})$$

## 2.9. Flatten và Fully Connected Layer

Sau nhiều lớp convolution và pooling, feature map được chuyển thành vector bằng Flatten. Vector này được đưa vào lớp Linear để thực hiện phân loại. Trong Basic CNN, sau hai lần pooling, feature map có dạng 32×7×7. Flatten chuyển nó thành 1568 giá trị, sau đó Linear tạo ra 10 logits tương ứng với mười lớp.

$$z = Wx + b$$

Logits chưa phải xác suất. Với CrossEntropyLoss, PyTorch nhận trực tiếp logits và nội bộ xử lý phần cần thiết cho phân loại nhiều lớp. Vì vậy, notebook không cần thêm Softmax trước CrossEntropyLoss.

## 2.10. CNN như một hàm hợp

Một CNN cơ bản có thể được mô tả bằng hàm hợp. Nếu ký hiệu Conv, ReLU, Pool, Flatten và Dense lần lượt là các phép biến đổi, ta có:

$$F(X) = Dense(Flatten(Pool(ReLU(Conv(X)))))$$

Điểm quan trọng của cách biểu diễn này là nó kết nối trực tiếp lý thuyết toán học với code. nn.Conv2d tương ứng với phép tích chập, nn.ReLU tương ứng với hàm phi tuyến, nn.MaxPool2d tương ứng với downsampling, nn.Flatten chuyển đổi dạng dữ liệu và nn.Linear thực hiện phép biến đổi cuối.

$$X \to Conv(X) \to ReLU(Conv(X)) \to Pool(\dots) \to Flatten(\dots) \to Dense(\dots)$$

*Hình 2. CNN được nhìn như một hàm hợp của nhiều phép biến đổi.*

## 2.11. Lan truyền thuận, hàm mất mát và tối ưu

Trong forward pass, dữ liệu đi từ input qua toàn bộ mạng để tạo output. Loss function đo mức độ khác biệt giữa dự đoán và nhãn thật. Backpropagation tính gradient của loss theo các tham số, sau đó optimizer cập nhật trọng số.

$$\theta \leftarrow \theta - \eta \ \nabla\theta \, L$$

Notebook sử dụng Adam với learning rate 0.001. Adam là optimizer kết hợp thông tin về gradient và moment để điều chỉnh bước cập nhật. Với MNIST và Fashion-MNIST, loss là CrossEntropyLoss. Với Diabetes, loss là BCEWithLogitsLoss vì đây là bài toán nhị phân.

Việc lưu cả training loss và validation loss ở từng epoch cho phép quan sát không chỉ khả năng học trên dữ liệu huấn luyện mà còn khả năng tổng quát hóa. Notebook chọn lại trạng thái model có validation accuracy tốt nhất trước khi đánh giá trên test set.

### 2.12. Train, validation và test

Ba tập dữ liệu có vai trò khác nhau. Training set dùng để cập nhật tham số. Validation set dùng để theo dõi và chọn trạng thái model trong quá trình phát triển. Test set được giữ độc lập để đánh giá cuối cùng.

Trong notebook, MNIST và Fashion-MNIST sử dụng 54.000 mẫu train, 6.000 mẫu validation và 10.000 mẫu test. Diabetes được chia theo tỷ lệ 70/15/15 với stratification để giữ phân bố lớp tương đối ổn định.

## 3. SỰ PHÁT TRIỂN CỦA CÁC KIẾN TRÚC CNN

### 3.1. Từ Basic CNN đến các kiến trúc sâu hơn

Notebook trình bày quá trình phát triển CNN theo góc nhìn vấn đề–giải pháp. Basic CNN cung cấp cơ chế học đặc trưng cục bộ. Khi mạng sâu hơn, các vấn đề về tối ưu, gradient, chi phí tính toán và khả năng biểu diễn xuất hiện. Các kiến trúc mới bổ sung những cơ chế nhằm xử lý từng vấn đề.

| Kiến trúc | Ý tưởng chính | Vấn đề hướng tới |
|---|---|---|
| LeNet | Convolution + pooling | Học mẫu không gian trong ảnh |
| AlexNet | Mạng sâu hơn, ReLU, dropout | Cải thiện phân loại ảnh quy mô lớn |
| VGG | Xếp chồng convolution 3×3 | Xây dựng biểu diễn sâu |
| Inception | Nhánh song song nhiều quy mô | Đặc trưng đa tỷ lệ |

| Kiến trúc | Ý tưởng chính | Vấn đề hướng tới |
|---|---|---|
| ResNet | Residual/skip connection | Tối ưu mạng sâu |
| DenseNet | Kết nối dày đặc | Tái sử dụng đặc trưng |
| MobileNet | Depthwise separable convolution | Giảm chi phí tính toán |
| EfficientNet | Compound scaling | Cân bằng depth/width/resolution |
| Attention | Gán trọng số cho feature | Tập trung vào đặc trưng quan trọng |

Trong Assignment 05, ba kiến trúc kinh điển LeNet, AlexNet và VGG được hiện thực dưới dạng lightweight adaptation cho ảnh 28×28. Điều này phù hợp với kích thước dữ liệu và giúp việc thực nghiệm vẫn nằm trong phạm vi của bài tập.

## 3.2. LeNet

LeNet là một trong những kiến trúc CNN kinh điển cho nhận dạng chữ số viết tay. Tinh thần của LeNet là kết hợp convolution, pooling và các lớp fully connected để học đặc trưng phân cấp.

Trong notebook, LeNet được điều chỉnh cho ảnh grayscale 28×28. Mạng dùng Conv2d 1→6 với kernel 5, ReLU, Average Pooling, sau đó Conv2d 6→16, tiếp tục pooling và cuối cùng là chuỗi Linear 120 → 84 → 10.

```
self.features = nn.Sequential(
    nn.Conv2d(1, 6, kernel_size=5),
    nn.ReLU(),
    nn.AvgPool2d(2),
    nn.Conv2d(6, 16, kernel_size=5),
    nn.ReLU(),
```

*nn.AvgPool2d(2)*

*)*

LeNet có 44.426 tham số trong phiên bản notebook. So với Basic CNN có 20.490 tham số, LeNet lớn hơn nhưng vẫn nhẹ hơn rất nhiều so với Adapted AlexNet.

### 3.3. AlexNet

AlexNet đại diện cho bước phát triển quan trọng của CNN sâu hơn, với ReLU và dropout đóng vai trò đáng kể trong huấn luyện mạng. Trong notebook, phiên bản thích nghi dùng ba convolution chính 1→32→64→128, pooling sau hai lớp đầu và classifier 128×7×7 → 256 → 10.

Do ảnh đầu vào chỉ 28×28, đây không phải kiến trúc AlexNet nguyên bản. Báo cáo gọi chính xác là Adapted AlexNet hoặc AlexNet-style CNN để tránh hiểu rằng notebook đã tái hiện kiến trúc ImageNet nguyên gốc.

### 3.4. VGG

VGG nổi bật bởi việc xây dựng mạng từ các convolution nhỏ 3×3 được xếp chồng. Trong notebook, Adapted VGG sử dụng hai cặp convolution 1→32→32 và 32→64→64, xen kẽ MaxPool2d, sau đó classifier 64×7×7 → 128 → 10.

Mô hình có 467.818 tham số. Kết quả cuối cùng trên MNIST đạt 99,30%, cao nhất trong bảy mô hình được đánh giá. Trên Fashion-MNIST, VGG đạt 90,81%, thấp hơn Adapted AlexNet 91,20%. Điều này cho thấy một kiến trúc không nhất thiết đứng đầu trên mọi dataset.

### 3.5. Inception, ResNet, DenseNet và các hướng phát triển khác

Notebook đưa Inception, ResNet, DenseNet, MobileNet, EfficientNet và Attention vào bảng tiến hóa để đặt các mô hình thực nghiệm trong bối cảnh rộng hơn. Inception dùng nhiều nhánh để thu nhận đặc trưng ở nhiều quy mô. ResNet dùng skip connection. DenseNet tái sử dụng đặc trưng bằng kết nối dày đặc. MobileNet hướng tới hiệu quả tính

toán. EfficientNet cân bằng nhiều chiều của quy mô mạng. Assignment 05 không triển khai tất cả các kiến trúc này nhằm đảm bảo phạm vi bài tập.

# 4. CƠ SỞ LÝ THUYẾT VỀ CÁC KỸ THUẬT CẢI TIẾN CNN

## 4.1. Batch Normalization

Batch Normalization chuẩn hóa các activation theo mini-batch và học thêm các tham số để điều chỉnh quy mô cũng như vị trí của phân phối. Trong notebook, BatchNorm2d được đặt sau convolution và trước ReLU.

$$\hat{x} = (x - \mu_B) / \sqrt{(\sigma_B^2 + \varepsilon)}$$

$$y = \gamma\hat{x} + \beta$$

Trong đó μ_B và σ_B² là trung bình và phương sai của mini-batch; γ và β là các tham số học được. Cách bố trí này giúp mạng kiểm soát tốt hơn phân phối activation trong quá trình huấn luyện.

```
nn.Conv2d(1, 16, kernel_size=3, padding=1),
nn.BatchNorm2d(16),
nn.ReLU(),
nn.MaxPool2d(2)
```

M2 đã được thực nghiệm là có 20.586 tham số, chỉ tăng nhẹ so với M1 20.490. Kết quả test cũng cho thấy BatchNorm cải thiện MNIST từ 98,72% lên 98,82% và Fashion-MNIST từ 89,46% lên 90,29%.

## 4.2. Residual Connection

Residual learning thay đổi cách mạng học ánh xạ. Thay vì buộc block phải học trực tiếp H(X), block học phần biến đổi F(X) rồi cộng với identity X.

$$Y = F(X) + X$$

Trong notebook, ResidualBlock gồm hai convolution 3×3, mỗi convolution đi kèm BatchNorm; ReLU được đặt sau convolution đầu và sau phép cộng. Identity được giữ nguyên và cộng vào đầu ra của convolution thứ hai.

*identity = x*
*out = self.conv1(x)*
*out = self.bn1(out)*
*out = self.relu(out)*
*out = self.conv2(out)*
*out = self.bn2(out)*
*out = out + identity*
*out = self.relu(out)*

M3 có 25.290 tham số. Trên MNIST, M3 đạt 99,04%, cao hơn M1 0,32 điểm phần trăm. Trên Fashion-MNIST, M3 đạt 90,78%, cao hơn M1 1,32 điểm phần trăm. Trên Diabetes, M3 đạt 86,21%, cao hơn M1 10,34 điểm phần trăm.

## 4.3. Ý nghĩa của identity path

Identity path tạo một đường truyền trực tiếp cho thông tin đầu vào. Về mặt khái niệm, block có thể giữ lại một phần biểu diễn ban đầu thay vì phải biến đổi toàn bộ thông tin qua các convolution. Đây là lý do residual connection thường được xem là cơ chế hỗ trợ tối ưu hóa mạng sâu.

## 4.4. Attention và Squeeze-and-Excitation

Notebook sử dụng channel attention theo cơ chế Squeeze-and-Excitation (SE). Với feature map X có C kênh, SE tạo một trọng số cho từng kênh rồi nhân trở lại với feature map.

$$Y = M_c(X) \odot X$$

Bước squeeze dùng AdaptiveAvgPool2d(1) để nén thông tin không gian thành một giá trị đại diện cho mỗi channel. Sau đó, hai lớp Linear với ReLU và Sigmoid tạo ra vector trọng số. Sigmoid đưa trọng số về khoảng 0–1.

```
y = self.pool(x).view(batch_size, channels)
weights = self.fc(y).view(batch_size, channels, 1, 1)
return x * weights
```

M4 kết hợp residual block và SE attention. M4 có 25.438 tham số. Trên MNIST, M4 đạt 99,00%; trên Fashion-MNIST đạt 90,92%. Như vậy attention không tự động tạo ra accuracy cao nhất trong mọi trường hợp.

## 4.5. Attention không đồng nghĩa với luôn tốt hơn

Kết quả thực nghiệm cho thấy M4 không đứng đầu MNIST hoặc Fashion-MNIST, mặc dù nó là mô hình có thêm cơ chế attention. Điều này là một kết quả có ý nghĩa: một kỹ thuật kiến trúc chỉ có giá trị khi phù hợp với dữ liệu, quy mô mạng và thiết lập huấn luyện. Việc thêm module cũng làm tăng chi phí tính toán.

## 4.6. Conv2d và Conv1d

Conv2d được sử dụng khi dữ liệu có hai chiều không gian rõ ràng. MNIST và Fashion-MNIST có dạng 1×28×28. Conv1d được sử dụng cho Diabetes sau khi tám đặc trưng được biểu diễn thành chuỗi 1×8.

$$X_image \in R^{(C \times H \times W)}$$

$$X_diabetes \in R^{(1 \times 8)}$$

Điểm cần nhấn mạnh là việc dùng Conv1d cho Diabetes trong notebook là một cách biểu diễn thực nghiệm để nghiên cứu CNN trên dữ liệu đặc trưng. Các thuộc tính bảng không mang cấu trúc không gian giống pixel ảnh, vì vậy kết quả không nên được diễn giải như bằng chứng rằng CNN luôn là lựa chọn tối ưu cho dữ liệu bảng.

## 4.7. Standardization và tránh rò rỉ dữ liệu

Diabetes được chuẩn hóa bằng StandardScaler. Notebook fit scaler trên training set rồi dùng scaler đó transform validation và test. Đây là quy trình đúng về mặt thực nghiệm vì thống kê từ test set không được dùng để xác định phép biến đổi của training data.

*scaler = StandardScaler()*

*X_train = scaler.fit_transform(X_train)*

*X_val = scaler.transform(X_val)*

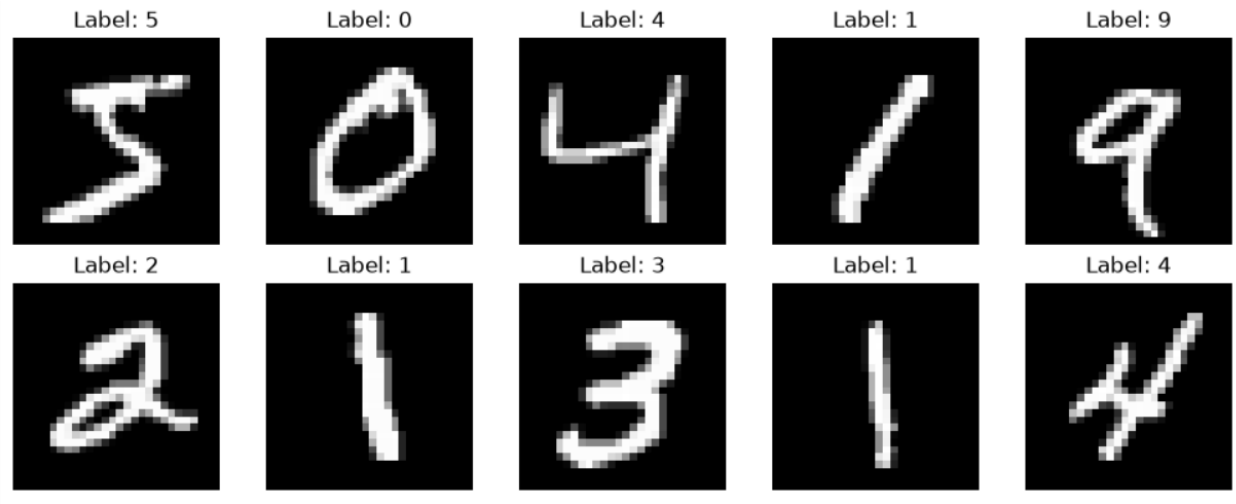
*X_test = scaler.transform(X_test)*

# 5. DỮ LIỆU VÀ BÀI TOÁN THỰC NGHIỆM

## 5.1. MNIST

MNIST là bộ dữ liệu chữ số viết tay gồm ảnh grayscale 28×28 và mười lớp từ 0 đến 9. Notebook sử dụng torchvision.datasets.MNIST và transforms.ToTensor(). Dữ liệu sau khi chuyển thành tensor có một channel.

Tập huấn luyện ban đầu có 60.000 mẫu và tập test có 10.000 mẫu. Notebook dành 6.000 mẫu từ training set làm validation, tạo thành 54.000 training, 6.000 validation và 10.000 test.




*Hình 3. Một số mẫu ảnh MNIST được trực quan hóa trong quá trình chuẩn bị dữ liệu.*

## 5.2. Dạng tensor và DataLoader

Batch size được đặt là 64. Training loader shuffle=True để thay đổi thứ tự mẫu giữa các epoch, trong khi validation và test loader dùng shuffle=False. Một batch ảnh có dạng [64, 1, 28, 28], còn nhãn có dạng [64].

```
BATCH_SIZE = 64

mnist_train_loader = DataLoader(
    mnist_train,
    batch_size=BATCH_SIZE,
    shuffle=True
)

mnist_test_loader = DataLoader(
    mnist_test,
    batch_size=BATCH_SIZE,
    shuffle=False
)

images, labels = next(iter(mnist_train_loader))

print("Batch image shape:", images.shape)
print("Batch label shape:", labels.shape)
```

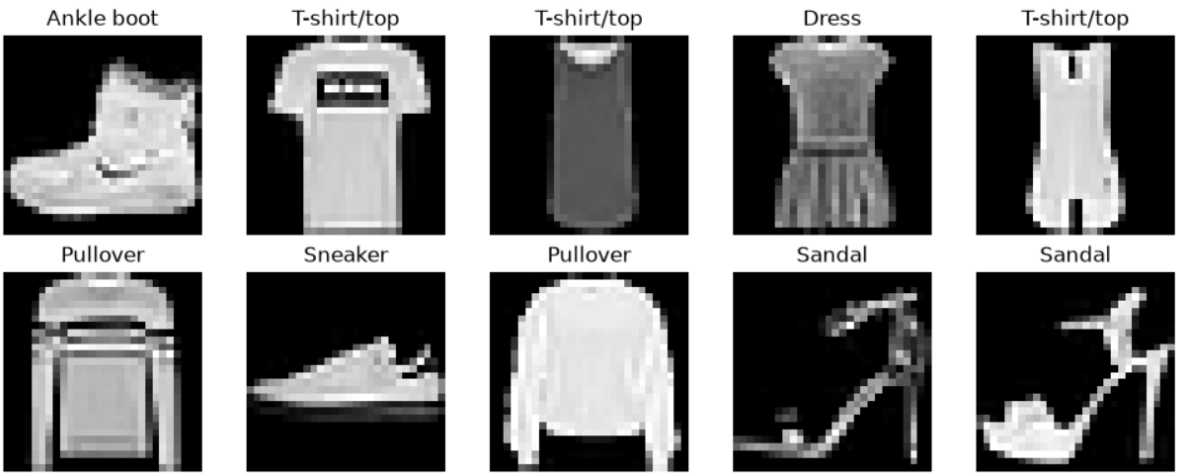
```
Batch image shape: torch.Size([64, 1, 28, 28])
Batch label shape: torch.Size([64])
```

*Hình 4. DataLoader của MNIST*

### 5.3. Fashion-MNIST

Fashion-MNIST giữ nguyên kích thước 1×28×28 và mười lớp nhưng thay nội dung chữ số bằng các đối tượng thời trang. Notebook khai báo tên mười lớp gồm T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag và Ankle boot.

Việc dùng cùng kích thước đầu vào giúp hai bộ dữ liệu ảnh được xử lý bằng cùng một nhóm kiến trúc. Do đó, sự khác biệt trong kết quả phản ánh một phần độ khó của dữ liệu thay vì thay đổi hoàn toàn thiết kế mạng.




*Hình 5. Một số mẫu Fashion-MNIST được trực quan hóa.*

Notebook chia 60.000 mẫu train thành 54.000 training và 6.000 validation, giữ nguyên 10.000 mẫu test. Cùng một seed 42 được dùng cho random split để tăng khả năng tái lập.

```
# Create a validation split from the Fashion-MNIST training dataset

VAL_SIZE = 6000
TRAIN_SIZE = len(fashion_train) - VAL_SIZE

fashion_train, fashion_val = random_split(
    fashion_train,
    [TRAIN_SIZE, VAL_SIZE],
    generator=torch.Generator().manual_seed(SEED)
)

fashion_train_loader = DataLoader(
    fashion_train,
    batch_size=BATCH_SIZE,
    shuffle=True
)

fashion_val_loader = DataLoader(
    fashion_val,
    batch_size=BATCH_SIZE,
    shuffle=False
)

print("Fashion-MNIST training samples  :", len(fashion_train))
print("Fashion-MNIST validation samples:", len(fashion_val))
print("Fashion-MNIST test samples      :", len(fashion_test))
```

```
Fashion-MNIST training samples  : 54000
Fashion-MNIST validation samples: 6000
Fashion-MNIST test samples      : 10000
```

*Hình 6. Training/Validation/Test samples của Fashion-Mnist*

## 5.4. Ý nghĩa của Fashion-MNIST trong thực nghiệm

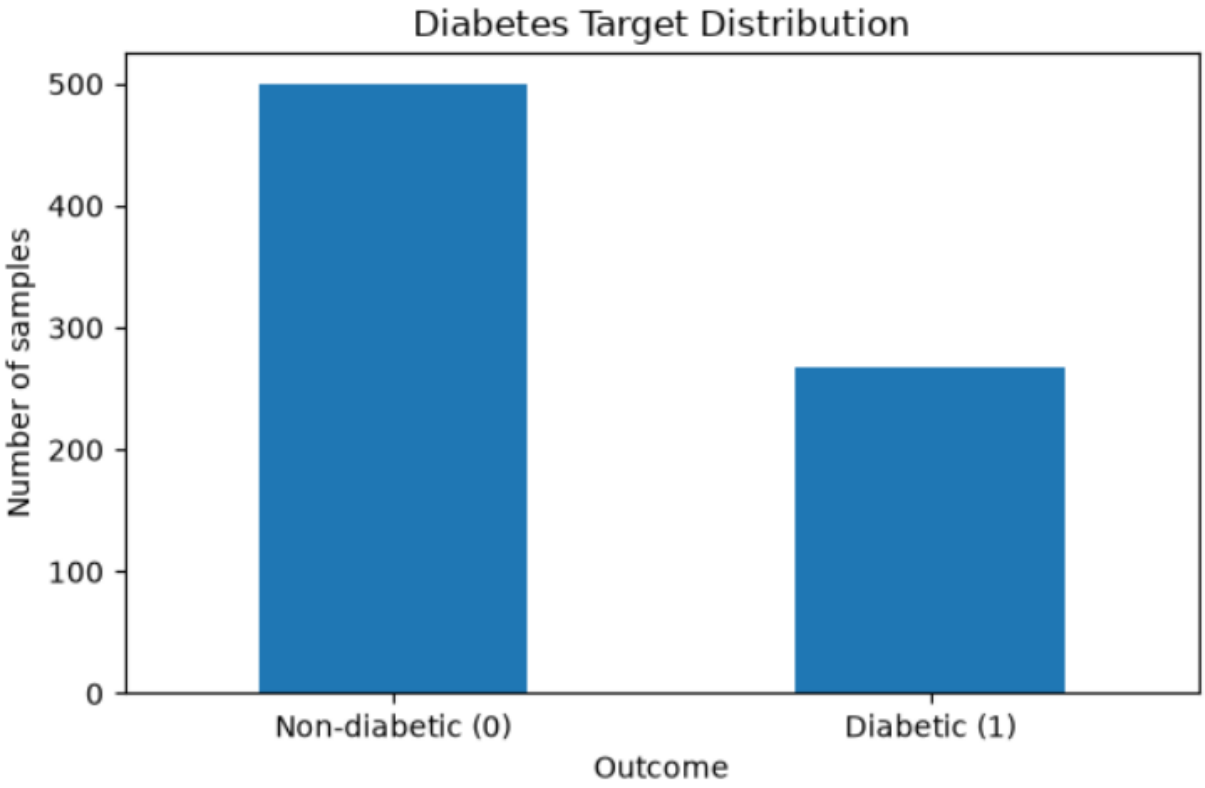
Fashion-MNIST đóng vai trò bài toán khó hơn MNIST. Các lớp có hình dạng tương tự nhau, chẳng hạn Shirt, T-shirt/top, Pullover và Coat, khiến việc phân loại dựa trên ảnh 28×28 khó hơn. Vì vậy, đây là dataset phù hợp để quan sát sự khác biệt giữa các kiến trúc CNN.

## 5.5. Diabetes dataset

Diabetes là bộ dữ liệu dạng bảng với 768 mẫu, tám thuộc tính đầu vào và biến Outcome nhị phân. Tám thuộc tính gồm Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction và Age.

| Đặc trưng | Vai trò |
|---|---|
| Pregnancies | Số lần mang thai |
| Glucose | Giá trị glucose |
| BloodPressure | Huyết áp |
| SkinThickness | Độ dày nếp gấp da |
| Insulin | Insulin |
| BMI | Chỉ số khối cơ thể |
| DiabetesPedigreeFunction | Chỉ số phả hệ tiểu đường |
| Age | Tuổi |
| Outcome | Mục tiêu: 0 hoặc 1 |

Notebook kiểm tra kích thước, thông tin cột, missing values, duplicate rows và phân bố target trước khi tiền xử lý. Một biểu đồ cột được sử dụng để trực quan hóa phân bố Outcome.




*Hình 7. Phân bố hai lớp Outcome của Diabetes.*

### 5.6. Xử lý giá trị 0

Notebook coi giá trị 0 trong Glucose, BloodPressure, SkinThickness, Insulin và BMI là giá trị cần xử lý. Các giá trị này được thay bằng NaN rồi điền bằng median của từng cột.

```python
diabetes_clean = diabetes_df.copy()

diabetes_clean[zero_as_missing] = diabetes_clean[zero_as_missing].replace(
    0, np.nan
)

for col in zero_as_missing:
    diabetes_clean[col] = diabetes_clean[col].fillna(
        diabetes_clean[col].median()
    )

print("Missing values after cleaning:")
print(diabetes_clean.isnull().sum())
```

```
Missing values after cleaning:
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

*Hình 8. Xử lý giá trị 0 trong Diabete*

## 5.7. Chia dữ liệu Diabetes

Dữ liệu Diabetes được chia thành training, validation và test theo tỷ lệ 70%, 15% và 15%. Hai lần train_test_split đều sử dụng stratify để duy trì phân bố lớp.

*768 → 537 train + 115 validation + 116 test*

Sau đó StandardScaler được fit trên training set. Validation và test chỉ được transform bằng scaler đã học. Vector tám chiều được unsqueeze thành tensor có dạng [batch, 1, 8].

```
X_train_tensor = torch.tensor(
    X_train, dtype=torch.float32
).unsqueeze(1)
```

DataLoader của Diabetes dùng batch size 32. Vì dataset nhỏ nên thời gian huấn luyện các mô hình Conv1d rất ngắn so với hai dataset ảnh.

## 5.8. So sánh đặc điểm ba dataset

| Dataset | Dạng dữ liệu | Kích thước input | Task | Số model |
|---|---|---|---|---|
| MNIST | Ảnh grayscale | 1×28×28 | 10 lớp | 7 |
| Fashion-MNIST | Ảnh grayscale | 1×28×28 | 10 lớp | 7 |
| Diabetes | Bảng số | 1×8 | Nhị phân | 4 |

Sự khác biệt về kích thước, dạng dữ liệu và số lượng mẫu là cơ sở để không so sánh accuracy giữa ba dataset theo nghĩa tuyệt đối. Mục tiêu chính là so sánh các mô hình bên trong từng dataset và quan sát tác động của kiến trúc.

```
print(f"MNIST:")
print(f"  Train: {len(mnist_train):,}")
print(f"  Test : {len(mnist_test):,}")
print(f"  Input: {mnist_train[0][0].shape}")

print(f"\nFashion-MNIST:")
print(f"  Train: {len(fashion_train):,}")
print(f"  Test : {len(fashion_test):,}")
print(f"  Input: {fashion_train[0][0].shape}")

print(f"\nDiabetes:")
print(f"  Train: {len(diabetes_train_dataset):,}")
print(f"  Val  : {len(diabetes_val_dataset):,}")
print(f"  Test : {len(diabetes_test_dataset):,}")
print(f"  Input: {X_train_tensor.shape[1:]}")
```

```
Dataset summary
============================================================
MNIST:
  Train: 54,000
  Test : 10,000
  Input: torch.Size([1, 28, 28])

Fashion-MNIST:
  Train: 54,000
  Test : 10,000
  Input: torch.Size([1, 28, 28])

Diabetes:
  Train: 537
  Val  : 115
  Test : 116
  Input: torch.Size([1, 8])
```

*Hình 9. Kết quả Dataset summary trong notebook*

# 6. THIẾT KẾ VÀ CÀI ĐẶT CÁC MÔ HÌNH CNN

## 6.1. Nhóm M1–M4

Bốn mô hình M1–M4 được thiết kế theo một tiến trình tăng dần. M1 là baseline. M2 bổ sung Batch Normalization. M3 bổ sung residual connection. M4 bổ sung channel attention SE trên nền residual CNN.

| Model | Thành phần chính | Parameters ảnh |
|---|---|---|
| M1 Basic CNN | Conv + ReLU + Pool | 20.490 |
| M2 BatchNorm | M1 + BatchNorm | 20.586 |
| M3 Residual | M2 + Residual | 25.290 |
| M4 Attention | M3 + SE attention | 25.438 |

Cách thiết kế này giúp thực nghiệm gần với một controlled comparison: các mô hình có cấu trúc tương đối gần nhau và mỗi bước thêm một cơ chế chính. Vì vậy, sự thay đổi kết quả có thể được thảo luận theo từng cơ chế thay vì chỉ so sánh các mạng hoàn toàn khác nhau.

## 6.2. M1 Basic CNN

M1 gồm Conv2d 1→16, ReLU, MaxPool2d, Conv2d 16→32, ReLU, MaxPool2d và Linear 32×7×7→10. Đây là baseline để làm mốc cho các cải tiến.

```python
# Define the basic CNN for image classification
class BasicCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()

        self.features = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(16, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(32 * 7 * 7, num_classes)
        )

    def forward(self, x):
        x = self.features(x)
        return self.classifier(x)
```

*Hình 10. Định nghĩa hàm cho M1*

### 6.3. M2 CNN với Batch Normalization

M2 giữ nguyên phần lớn cấu trúc của M1 và chèn BatchNorm2d sau convolution. Mục tiêu là ổn định activation và hỗ trợ quá trình huấn luyện.

```python
# Define a CNN with Batch Normalization
class CNNBatchNorm(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()

        self.features = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1),
            nn.BatchNorm2d(16),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(16, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(32 * 7 * 7, num_classes)
        )

    def forward(self, x):
        x = self.features(x)
        return self.classifier(x)
```

*Hình 11. Định nghĩa hàm cho M2*

```python
# Compare trainable parameters between M1 and M2
print("M1 parameters:", count_parameters(m1))
print("M2 parameters:", count_parameters(m2))
print("Additional parameters:",
      count_parameters(m2) - count_parameters(m1))
```

```
M1 parameters: 20490
M2 parameters: 20586
Additional parameters: 96
```

*Hình 12. So sánh tham số giữa M1 và M2*

M2 chỉ tăng 96 tham số so với M1 nhưng cho kết quả test cao hơn trên cả MNIST và Fashion-MNIST. Đây là ví dụ trực quan về một cải tiến nhỏ về số tham số nhưng có thể tạo thay đổi trong quá trình học.

## 6.4. M3 Residual CNN

M3 gồm BatchNorm và một ResidualBlock với hai convolution cùng số channel. Identity được cộng trực tiếp vào output của block.

```python
# Define a residual block for the CNN
class ResidualBlock(nn.Module):
    def __init__(self, channels):
        super().__init__()

        self.conv1 = nn.Conv2d(
            channels, channels,
            kernel_size=3,
            padding=1
        )

        self.bn1 = nn.BatchNorm2d(channels)

        self.conv2 = nn.Conv2d(
            channels, channels,
            kernel_size=3,
            padding=1
        )

        self.bn2 = nn.BatchNorm2d(channels)

        self.relu = nn.ReLU()

    def forward(self, x):
        identity = x

        out = self.conv1(x)
        out = self.bn1(out)
        out = self.relu(out)

        out = self.conv2(out)
        out = self.bn2(out)

        out = out + identity
        out = self.relu(out)

        return out
```

*Hình 13. Định nghĩa ResidualBlock*

```python
# Define the CNN with a residual connection
class ResidualCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()

        self.conv1 = nn.Conv2d(
            1, 16,
            kernel_size=3,
            padding=1
        )

        self.bn1 = nn.BatchNorm2d(16)

        self.residual = ResidualBlock(16)

        self.pool1 = nn.MaxPool2d(2)

        self.conv2 = nn.Conv2d(
            16, 32,
            kernel_size=3,
            padding=1
        )

        self.bn2 = nn.BatchNorm2d(32)

        self.relu = nn.ReLU()
        self.pool2 = nn.MaxPool2d(2)

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(32 * 7 * 7, num_classes)
        )

    def forward(self, x):
        x = self.conv1(x)
        x = self.bn1(x)
        x = self.relu(x)

        x = self.residual(x)
        x = self.pool1(x)

        x = self.conv2(x)
        x = self.bn2(x)
        x = self.relu(x)
        x = self.pool2(x)

        return self.classifier(x)
```

*Hình 14. Định nghĩa ResidualCNN*

Trong phiên bản ảnh, M3 có 25.290 tham số. Trên MNIST, accuracy đạt 99,04%. Trên Fashion-MNIST, accuracy đạt 90,78%. Trên Diabetes, phiên bản Conv1d đạt 86,21%.

## 6.5. M4 Attention CNN

M4 đặt SEBlock sau residual block. SEBlock học mức độ quan trọng của từng channel. Với 16 channel và reduction=4, hidden layer có kích thước 4.

```python
# Define a channel attention block using Squeeze-and-Excitation
class SEBlock(nn.Module):
    def __init__(self, channels, reduction=4):
        super().__init__()

        hidden = max(channels // reduction, 1)

        self.pool = nn.AdaptiveAvgPool2d(1)

        self.fc = nn.Sequential(
            nn.Linear(channels, hidden),
            nn.ReLU(),
            nn.Linear(hidden, channels),
            nn.Sigmoid()
        )

    def forward(self, x):
        batch_size, channels, _, _ = x.shape

        # Squeeze spatial information into one value per channel
        y = self.pool(x).view(batch_size, channels)

        # Learn channel importance weights
        weights = self.fc(y).view(
            batch_size,
            channels,
            1,
            1
        )

        # Reweight feature channels
        return x * weights
```

*Hình 15. Định nghĩa SEBlock*

```python
class AttentionCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()

        self.conv1 = nn.Conv2d(
            1, 16,
            kernel_size=3,
            padding=1
        )

        self.bn1 = nn.BatchNorm2d(16)

        self.residual = ResidualBlock(16)

        self.attention = SEBlock(16)

        self.pool1 = nn.MaxPool2d(2)

        self.conv2 = nn.Conv2d(
            16, 32,
            kernel_size=3,
            padding=1
        )

        self.bn2 = nn.BatchNorm2d(32)

        self.relu = nn.ReLU()
        self.pool2 = nn.MaxPool2d(2)

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(32 * 7 * 7, num_classes)
        )

    def forward(self, x):
        x = self.conv1(x)
        x = self.bn1(x)
        x = self.relu(x)

        x = self.residual(x)

        x = self.attention(x)

        x = self.pool1(x)

        x = self.conv2(x)
        x = self.bn2(x)
        x = self.relu(x)
        x = self.pool2(x)

        return self.classifier(x)
```

*Hình 16. Định nghĩa AttentionCNN*

M4 có 25.438 tham số đối với ảnh và 3.637 tham số đối với Diabetes. Dù là mô hình giàu cơ chế nhất trong nhóm M1–M4, nó không đạt accuracy cao nhất trên cả ba dataset.

## 6.6. Mô hình CNN cho Diabetes

Do input Diabetes có dạng 1×8, notebook thay Conv2d bằng Conv1d. M1 dùng Conv1d 1→16, MaxPool1d, Conv1d 16→32 và Linear 32×4→1. M2 thêm BatchNorm1d. M3 dùng ResidualBlock1D. M4 dùng SEBlock1D.

| Model | Parameters | Ý tưởng |
|---|---|---|
| M1 Basic CNN | 1.761 | Baseline Conv1d |
| M2 BatchNorm | 1.857 | Chuẩn hóa activation |
| M3 Residual | 3.489 | Skip connection |
| M4 Attention | 3.637 | Residual + SE |

```python
# Define the basic 1D CNN for the Diabetes dataset
class DiabetesBasicCNN(nn.Module):
    def __init__(self):
        super().__init__()

        self.features = nn.Sequential(
            nn.Conv1d(1, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool1d(2),

            nn.Conv1d(16, 32, kernel_size=3, padding=1),
            nn.ReLU()
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(32 * 4, 1)
        )

    def forward(self, x):
        x = self.features(x)
        return self.classifier(x).squeeze(1)
```

*Hình 17. Định nghĩa DiabetesBasicCNN*

```python
# Define the Diabetes CNN with Batch Normalization
class DiabetesBatchNormCNN(nn.Module):
    def __init__(self):
        super().__init__()

        self.features = nn.Sequential(
            nn.Conv1d(1, 16, kernel_size=3, padding=1),
            nn.BatchNorm1d(16),
            nn.ReLU(),
            nn.MaxPool1d(2),

            nn.Conv1d(16, 32, kernel_size=3, padding=1),
            nn.BatchNorm1d(32),
            nn.ReLU()
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(32 * 4, 1)
        )

    def forward(self, x):
        x = self.features(x)
        return self.classifier(x).squeeze(1)
```

*Hình 18. Định nghĩa hàm BatchNorm cho Diabete*

```python
# Define a residual block for one-dimensional feature sequences
class ResidualBlock1D(nn.Module):
    def __init__(self, channels):
        super().__init__()

        self.conv1 = nn.Conv1d(
            channels, channels,
            kernel_size=3,
            padding=1
        )

        self.bn1 = nn.BatchNorm1d(channels)

        self.conv2 = nn.Conv1d(
            channels, channels,
            kernel_size=3,
            padding=1
        )

        self.bn2 = nn.BatchNorm1d(channels)

        self.relu = nn.ReLU()

    def forward(self, x):
        identity = x

        out = self.conv1(x)
        out = self.bn1(out)
        out = self.relu(out)

        out = self.conv2(out)
        out = self.bn2(out)

        out = out + identity
        out = self.relu(out)

        return out
```

*Hình 19. Định nghĩa ResidualBlock1D*

```python
# Define the Diabetes CNN with a residual connection
class DiabetesResidualCNN(nn.Module):
    def __init__(self):
        super().__init__()

        self.conv1 = nn.Conv1d(
            1, 16,
            kernel_size=3,
            padding=1
        )

        self.bn1 = nn.BatchNorm1d(16)
        self.relu = nn.ReLU()

        self.residual = ResidualBlock1D(16)
        self.pool = nn.MaxPool1d(2)

        self.conv2 = nn.Conv1d(
            16, 32,
            kernel_size=3,
            padding=1
        )

        self.bn2 = nn.BatchNorm1d(32)

        self.classifier = nn.Linear(32 * 4, 1)

    def forward(self, x):
        x = self.conv1(x)
        x = self.bn1(x)
        x = self.relu(x)

        x = self.residual(x)
        x = self.pool(x)

        x = self.conv2(x)
        x = self.bn2(x)
        x = self.relu(x)

        return self.classifier(
            x.flatten(start_dim=1)
        ).squeeze(1)
```

*Hình 20. Định nghĩa DiabetesResidualCNN*

```python
# Define a channel attention block for one-dimensional features
class SEBlock1D(nn.Module):
    def __init__(self, channels, reduction=4):
        super().__init__()

        hidden = max(channels // reduction, 1)

        self.pool = nn.AdaptiveAvgPool1d(1)

        self.fc = nn.Sequential(
            nn.Linear(channels, hidden),
            nn.ReLU(),
            nn.Linear(hidden, channels),
            nn.Sigmoid()
        )

    def forward(self, x):
        batch_size, channels, _ = x.shape

        # Squeeze the sequence dimension
        y = self.pool(x).view(batch_size, channels)

        # Learn channel importance weights
        weights = self.fc(y).view(
            batch_size,
            channels,
            1
        )

        # Reweight feature channels
        return x * weights
```

*Hình 21. Định nghĩa SEBlock1D cho Diabetes*

```python
# Define the Diabetes CNN with residual connection and attention
class DiabetesAttentionCNN(nn.Module):
    def __init__(self):
        super().__init__()

        self.conv1 = nn.Conv1d(
            1, 16,
            kernel_size=3,
            padding=1
        )

        self.bn1 = nn.BatchNorm1d(16)
        self.relu = nn.ReLU()

        self.residual = ResidualBlock1D(16)
        self.attention = SEBlock1D(16)

        self.pool = nn.MaxPool1d(2)

        self.conv2 = nn.Conv1d(
            16, 32,
            kernel_size=3,
            padding=1
        )

        self.bn2 = nn.BatchNorm1d(32)

        self.classifier = nn.Linear(32 * 4, 1)

    def forward(self, x):
        x = self.conv1(x)
        x = self.bn1(x)
        x = self.relu(x)

        x = self.residual(x)
        x = self.attention(x)

        x = self.pool(x)

        x = self.conv2(x)
        x = self.bn2(x)
        x = self.relu(x)

        return self.classifier(
            x.flatten(start_dim=1)
        ).squeeze(1)
```

*Hình 22. Định nghĩa DiabetesAttentionCNN*

Các mô hình Diabetes có ít tham số hơn đáng kể so với mô hình ảnh. Tuy nhiên, số tham số thấp không có nghĩa bài toán dễ hơn; kích thước dataset nhỏ và bản chất tabular làm kết quả validation/test nhạy hơn.

## 6.7. Classical CNN – LeNet, AlexNet và VGG

Ba mô hình kinh điển được hiện thực dưới dạng thích nghi. LeNet có 44.426 tham số, Adapted AlexNet có 1.701.130 tham số và Adapted VGG có 467.818 tham số.

Việc gọi các mô hình là 'adapted' rất quan trọng vì cấu hình trong notebook được điều chỉnh cho ảnh 28×28 grayscale, không phải cấu hình gốc của các bài toán quy mô lớn.

## 6.8. Kiểm tra output shape và parameter count

Trước khi huấn luyện, notebook dùng dummy input để kiểm tra output shape của các model. Đây là bước xác minh quan trọng để phát hiện lỗi kích thước tensor trước khi bước vào huấn luyện tốn thời gian.

```
test_input = torch.randn(4, 1, 28, 28)


for name, model in models.items():
    model.eval()
    with torch.no_grad():
        output = model(test_input)
    print(name, output.shape)
```

Notebook cũng định nghĩa hàm count_parameters để đếm số tham số trainable. Việc này cho phép báo cáo không chỉ nêu accuracy mà còn thảo luận về độ phức tạp của mô hình.

$$Parameters = \Sigma\ numel(parameter),\ với\ requires_grad = True$$

## 6.9. Tổng hợp kiến trúc cuối cùng

| Dataset | M1 | M2 | M3 | M4 |
|---|---|---|---|---|
| MNIST | Basic | + BN | + Residual | + Attention |
| Fashion-MNIST | Basic | + BN | + Residual | + Attention |

| Dataset | M1 | M2 | M3 | M4 |
|---|---|---|---|---|
| Diabetes | Basic | + BN | + Residual | + Attention |

# 7. QUY TRÌNH HUẤN LUYỆN VÀ ĐÁNH GIÁ

## 7.1. Thiết lập huấn luyện

Notebook đặt SEED = 42 cho Python random, NumPy và PyTorch. Image models được huấn luyện 5 epoch, Diabetes models 15 epoch, learning rate 0,001 và optimizer Adam.

| Thiết lập | Giá trị |
|---|---|
| Seed | 42 |
| Image epochs | 5 |
| Diabetes epochs | 15 |
| Learning rate | 0.001 |
| Image loss | CrossEntropyLoss |
| Diabetes loss | BCEWithLogitsLoss |
| Optimizer | Adam |

Mỗi model được tạo mới trước khi train để tránh việc tham số của một thực nghiệm ảnh hưởng đến thực nghiệm tiếp theo. Trong training function, history của train loss, validation loss, train accuracy và validation accuracy được lưu lại.

## 7.2. Lựa chọn best state

Sau mỗi epoch, validation accuracy được kiểm tra. Trạng thái model tốt nhất được lưu bằng copy.deepcopy. Khi kết thúc training, model được load lại best_state. Cách làm

này giúp test set được đánh giá bằng trạng thái tốt nhất theo validation thay vì đơn giản là trạng thái ở epoch cuối.

### 7.3. Đánh giá bằng test set

Section 7 của notebook đánh giá toàn bộ 18 thực nghiệm: 7 model trên MNIST, 7 model trên Fashion-MNIST và 4 model trên Diabetes. Test set chỉ được dùng ở bước inference cuối.

Với image classification, model tạo logits và chọn class có logit lớn nhất bằng argmax. Với Diabetes, logits được đưa qua sigmoid và threshold 0,5 để tạo dự đoán 0/1.

```
probabilities = torch.sigmoid(outputs)
predictions = (probabilities >= 0.5).float()
```

### 7.4. Metrics

Accuracy đo tỷ lệ dự đoán đúng trên tổng số mẫu. Precision cho biết trong các mẫu được dự đoán là một lớp, tỷ lệ dự đoán đúng là bao nhiêu. Recall cho biết trong các mẫu thực sự thuộc lớp, mô hình nhận diện đúng bao nhiêu. F1-score là trung bình điều hòa của Precision và Recall.

$$Accuracy = (TP + TN) / (TP + TN + FP + FN)$$

$$Precision = TP / (TP + FP)$$

$$Recall = TP / (TP + FN)$$

$$F1 = 2 \times Precision \times Recall / (Precision + Recall)$$

Notebook dùng weighted average cho Precision, Recall và F1-score nhằm tính đến phân bố số lượng mẫu giữa các lớp.

## 8. KẾT QUẢ THỰC NGHIỆM

### 8.1. Kết quả MNIST

| Model | Parameters | Time (s) | Test Loss | Accuracy |
|---|---|---|---|---|
| M1 Basic CNN | 20,490 | 256.59 | 0.039796 | 98.72 |
| M2 BatchNorm | 20,586 | 289.47 | 0.036356 | 98.82 |
| M3 Residual | 25,290 | 541.19 | 0.028206 | 99.04 |
| M4 Attention | 25,438 | 622.51 | 0.031412 | 99.00 |
| M5 LeNet | 44,426 | 190.04 | 0.048912 | 98.50 |
| M6 Adapted AlexNet | 1,701,130 | 382.06 | 0.025842 | 99.19 |
| M7 Adapted VGG | 467,818 | 343.75 | 0.022031 | 99.30 |

M7 Adapted VGG đạt accuracy cao nhất 99,30% và test loss thấp nhất 0,022031 trong nhóm MNIST. M6 Adapted AlexNet đạt 99,19% với số tham số lớn nhất. M3 Residual đạt 99,04%, cho thấy residual connection đã cải thiện baseline rõ ràng.

M5 LeNet có thời gian huấn luyện thấp nhất trong nhóm image nhưng accuracy 98,50%, thấp nhất trong bảy mô hình. Kết quả cho thấy tốc độ và accuracy cần được xem xét đồng thời.

## 8.2. Kết quả Fashion-MNIST

| Model | Parameters | Time (s) | Test Loss | Accuracy |
|---|---|---|---|---|
| M1 Basic CNN | 20,490 | 88.13 | 0.303449 | 89.46 |
| M2 BatchNorm | 20,586 | 103.22 | 0.279555 | 90.29 |
| M3 Residual | 25,290 | 190.37 | 0.253040 | 90.78 |
| M4 Attention | 25,438 | 203.07 | 0.255394 | 90.92 |

| Model | Parameters | Time (s) | Test Loss | Accuracy |
|---|---|---|---|---|
| M5 LeNet | 44,426 | 59.71 | 0.385783 | 86.02 |
| M6 Adapted AlexNet | 1,701,130 | 246.36 | 0.242934 | 91.20 |
| M7 Adapted VGG | 467,818 | 346.91 | 0.242247 | 90.81 |

Fashion-MNIST khó hơn MNIST. Accuracy của baseline M1 là 89,46%, trong khi model tốt nhất M6 Adapted AlexNet đạt 91,20%, cải thiện 1,74 điểm phần trăm.

M4 Attention đạt 90,92%, cao hơn M3 Residual 0,14 điểm phần trăm. M7 VGG đạt 90,81%. LeNet đạt 86,02%, cho thấy kiến trúc nhẹ và cổ điển này có giới hạn rõ hơn trên bài toán thời trang.

Điểm đáng chú ý là test loss của M7 là 0,242247, thấp nhất nhóm Fashion-MNIST, nhưng accuracy của M6 lại cao hơn. Điều này cho thấy việc đánh giá mô hình không nên dựa trên một metric duy nhất.

## 8.3. Kết quả Diabetes

| Model | Parameters | Time (s) | Test Loss | Accuracy |
|---|---|---|---|---|
| M1 Basic CNN | 1,761 | 0.75 | 0.493523 | 75.86 |
| M2 BatchNorm | 1,857 | 0.95 | 0.374922 | 82.76 |
| M3 Residual | 3,489 | 1.43 | 0.389033 | 86.21 |
| M4 Attention | 3,637 | 1.97 | 0.397135 | 81.90 |

M3 Residual đạt accuracy cao nhất 86,21%, vượt M1 Basic CNN 10,34 điểm phần trăm. M2 BatchNorm đạt 82,76%, còn M4 Attention đạt 81,90%. M4 có nhiều tham số và thời gian huấn luyện dài nhất trong nhóm nhưng không đạt accuracy cao nhất.

Kết quả này đặc biệt quan trọng về mặt phân tích: thêm cơ chế attention không đảm bảo cải thiện khi dataset nhỏ. M3 Residual có sự cân bằng tốt hơn trong thực nghiệm cụ thể này.

## 8.4. Classification metrics

| Dataset | Model | Accuracy | Precision | Recall | F1 |
|---|---|---|---|---|---|
| MNIST | M7 Adapted VGG | 99.30% | 99.30% | 99.30% | 99.30% |
| Fashion-MNIST | M6 Adapted AlexNet | 91.20% | 91.16% | 91.20% | 91.15% |
| Diabetes | M3 Residual | 86.21% | 86.69% | 86.21% | 85.68% |

Riêng Diabetes, F1-score 85,68% thấp hơn accuracy 86,21%, nhấn mạnh rằng accuracy không phản ánh toàn bộ hành vi phân loại. Precision 86,69% cho thấy tỷ lệ dự đoán dương đúng ở mức tương đối cao trong weighted calculation.

## 8.5. Accuracy improvement so với Basic CNN

| Dataset | Model được chọn | Baseline | Best | Improvement |
|---|---|---|---|---|
| MNIST | M7 Adapted VGG | 98.72% | 99.30% | +0.58 pp |
| Fashion-MNIST | M6 Adapted AlexNet | 89.46% | 91.20% | +1.74 pp |

| Dataset | Model được chọn | Baseline | Best | Improvement |
|---|---|---|---|---|
| Diabetes | M3 Residual | 75.86% | 86.21% | +10.34 pp |

Sự cải thiện trên MNIST tương đối nhỏ vì baseline đã đạt mức rất cao. Khi một dataset dễ và accuracy đã gần 100%, dư địa cải thiện bị giới hạn. Fashion-MNIST cho thấy khoảng cách lớn hơn giữa baseline và các kiến trúc cải tiến.

Diabetes có mức cải thiện lớn nhất về mặt điểm phần trăm. Tuy nhiên, cần thận trọng khi diễn giải vì đây là dataset nhỏ và bài toán Conv1d được xây dựng như một biểu diễn thực nghiệm của dữ liệu bảng.

### 8.6. Complexity

Số tham số dao động từ 20.490 của M1 ảnh đến 1.701.130 của Adapted AlexNet. M6 là mô hình có số tham số lớn nhất. M4 có 25.438 tham số, chỉ cao hơn M3 148 tham số nhưng thời gian huấn luyện MNIST tăng từ 541,19 lên 622,51 giây.

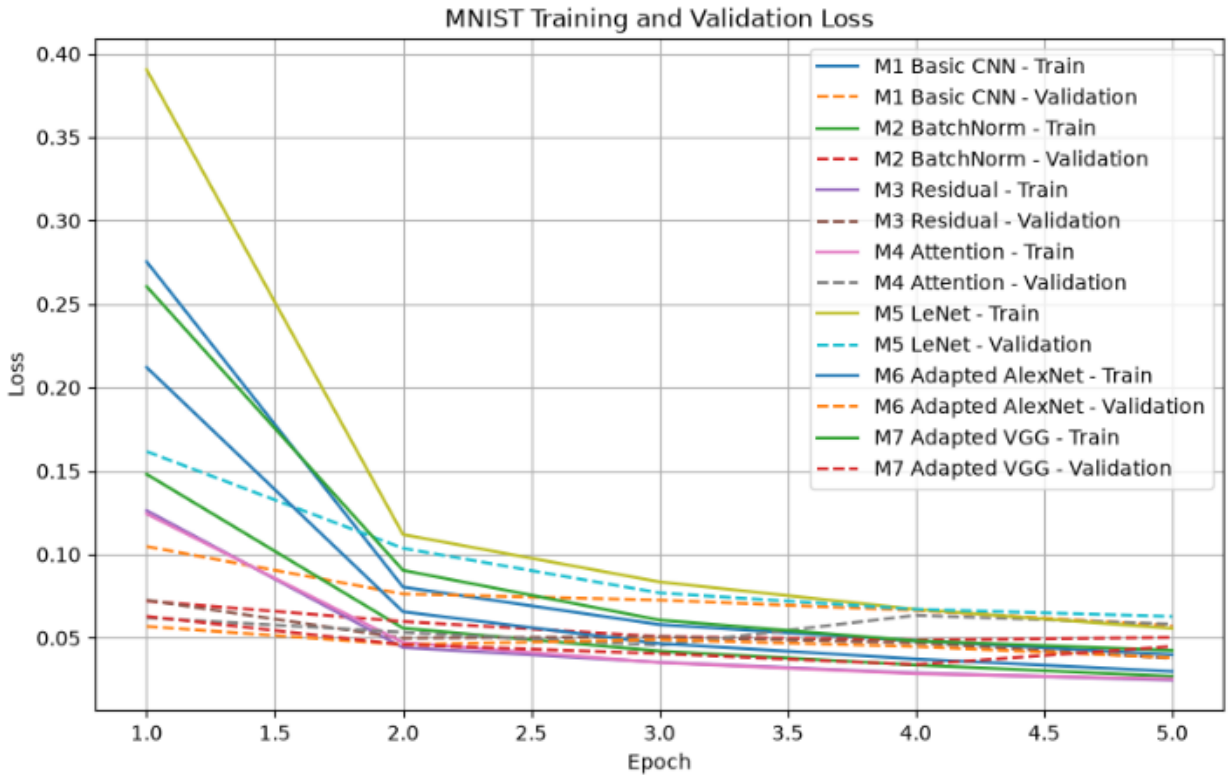
Điều này minh họa rằng số tham số là một chỉ báo quan trọng nhưng không phải yếu tố duy nhất quyết định thời gian. Các phép toán bổ sung và cấu trúc forward cũng ảnh hưởng đến chi phí.

## 9. TRỰC QUAN HÓA VÀ PHÂN TÍCH BIỂU ĐỒ

### 9.1. Vai trò của visualization

Visualization giúp chuyển các con số trong bảng thành các mẫu dễ quan sát. Phần này không chỉ trình bày ảnh biểu đồ mà còn giải thích xu hướng, điểm bất thường và mối quan hệ giữa accuracy, loss, training time và số tham số.

## 9.2. Training và validation loss – MNIST

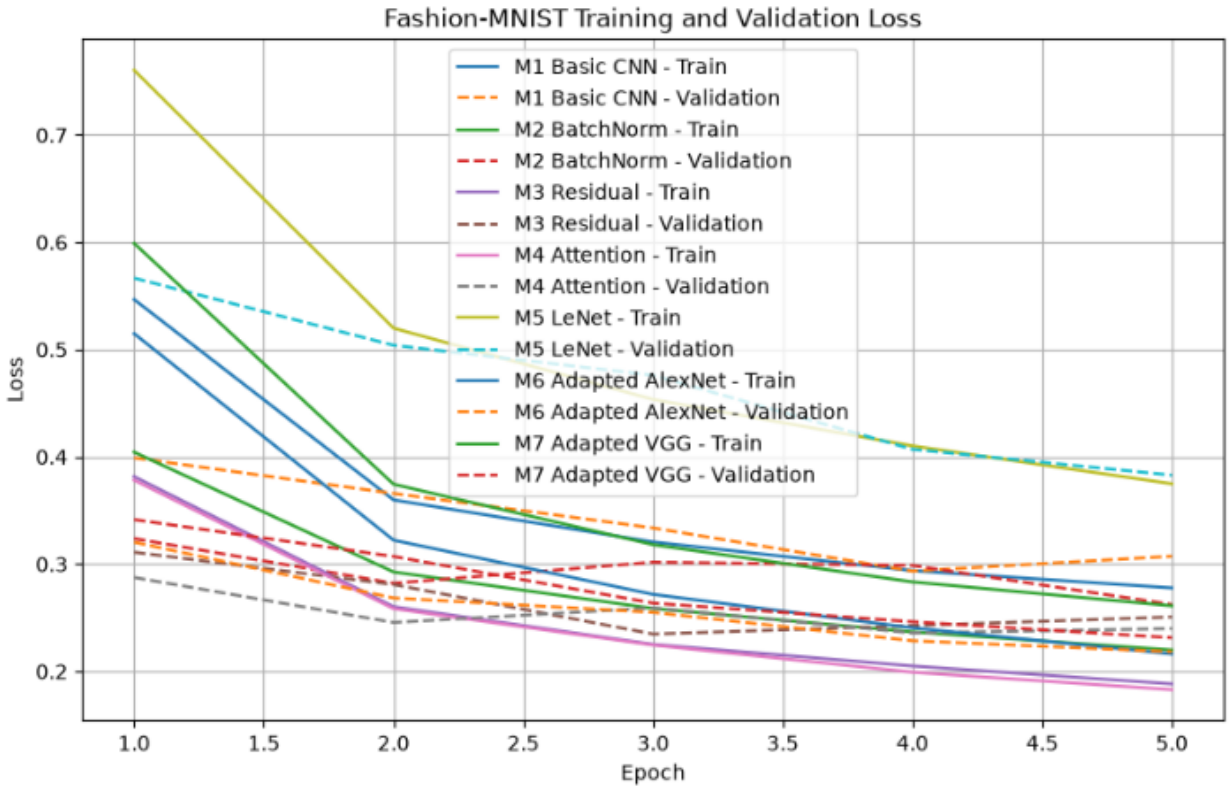



*Hình 23. Đường loss huấn luyện và validation của bảy mô hình trên MNIST.*

Biểu đồ cần được đọc theo hai hướng. Thứ nhất, train loss giảm dần thể hiện mô hình học được từ dữ liệu. Thứ hai, khoảng cách giữa train loss và validation loss phản ánh khả năng tổng quát hóa. Với MNIST, các đường loss nhìn chung giảm nhanh, phù hợp với kết quả accuracy cao.

M3 Residual và M7 Adapted VGG có test loss thấp trong nhóm. Tuy nhiên, đường validation không nhất thiết giảm đơn điệu ở mọi epoch. Điều này là bình thường trong huấn luyện deep learning và là lý do notebook lưu best validation state.

## 9.3. Training và validation loss – Fashion-MNIST

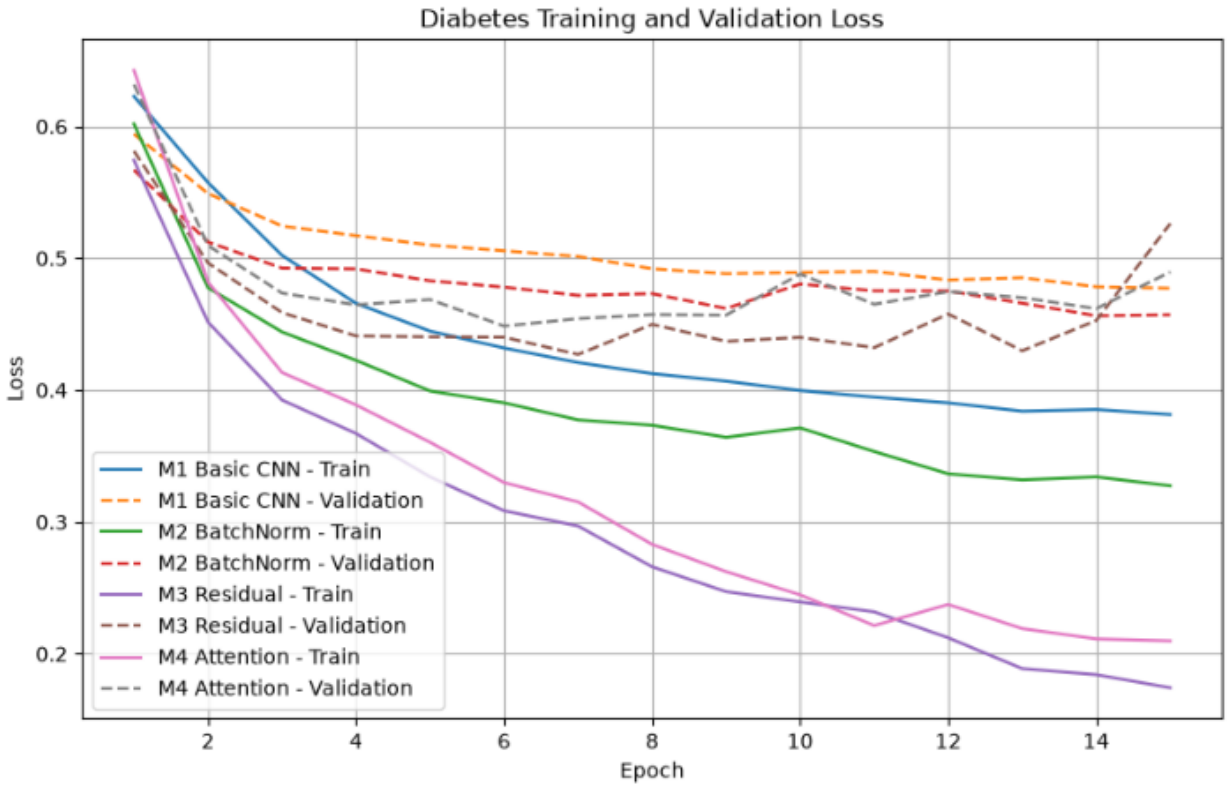



*Hình 24. Đường loss huấn luyện và validation của các mô hình trên Fashion-MNIST.*

Fashion-MNIST có loss cao hơn MNIST và validation loss giảm chậm hơn. Đây là biểu hiện phù hợp với độ khó cao hơn của dữ liệu. M6 Adapted AlexNet đạt test accuracy cao nhất 91,20%, trong khi M7 có test loss thấp nhất 0,242247.

Khi phân tích biểu đồ, cần tránh kết luận rằng đường train loss thấp nhất luôn đồng nghĩa với model tốt nhất. Model cần tổng quát hóa tốt trên validation/test. Đây là lý do báo cáo sử dụng cả loss và các metric phân loại.

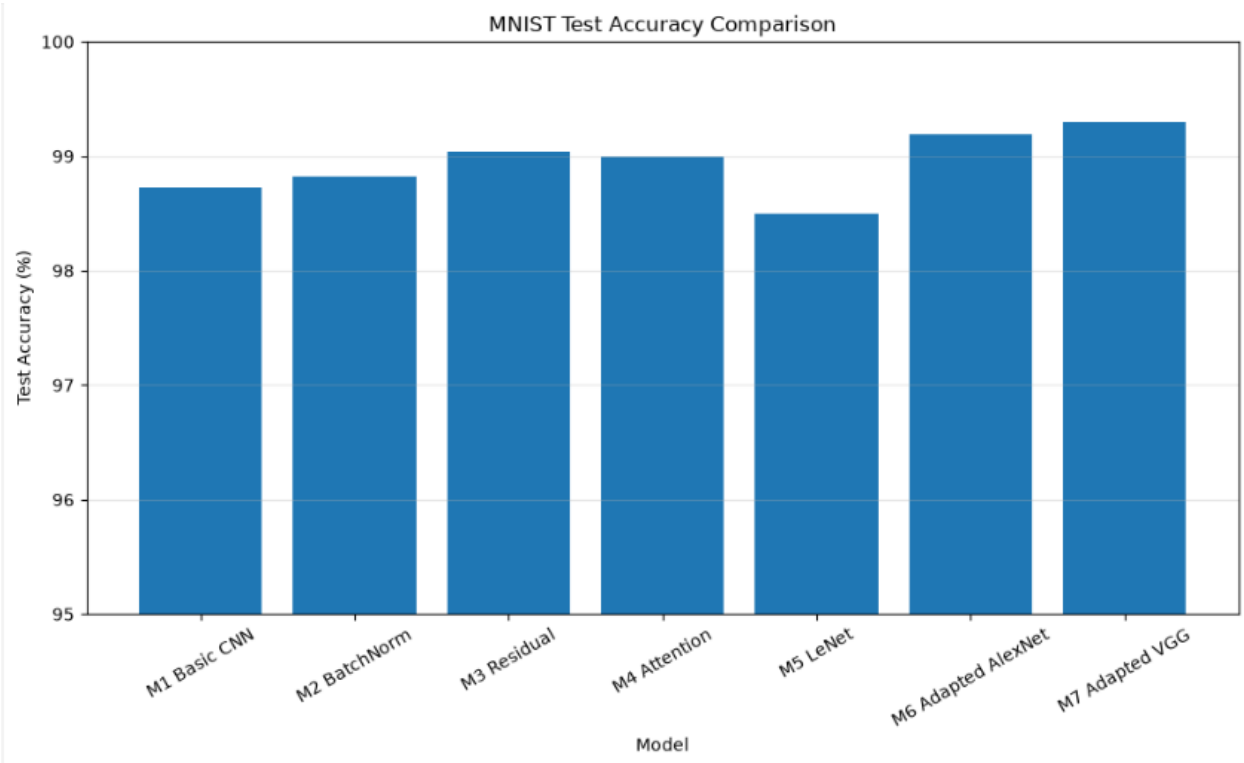
## 9.4. Training và validation loss – Diabetes




*Hình 25. Đường loss huấn luyện và validation của bốn mô hình Diabetes.*

Diabetes có số lượng mẫu nhỏ nên đường validation loss có thể dao động mạnh hơn so với hai dataset ảnh. M3 có train loss giảm đáng kể, nhưng test loss cuối cùng không thấp nhất. Điều này nhấn mạnh sự khác biệt giữa tối ưu training objective và kết quả phân loại trên test set.

## 9.5. Test accuracy – MNIST

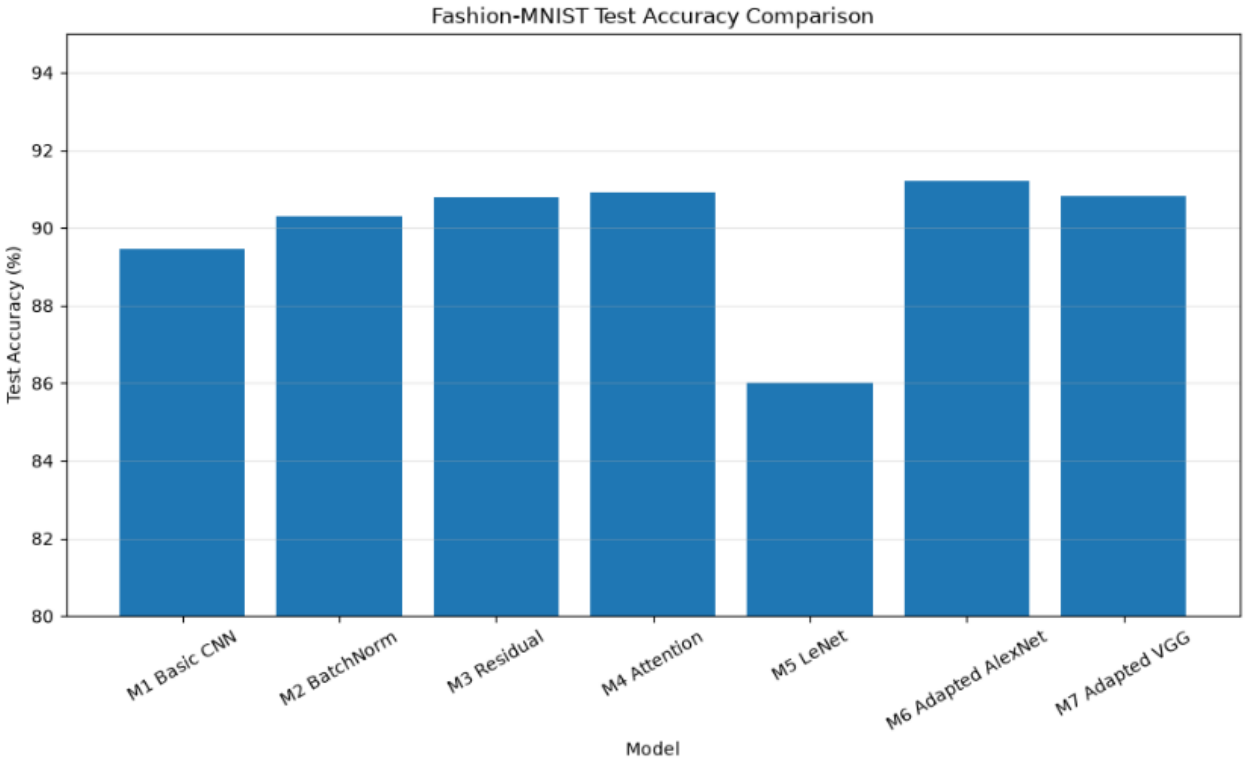



*Hình 26. So sánh test accuracy của bảy mô hình trên MNIST.*

Biểu đồ cột cho thấy tất cả mô hình đều đạt accuracy cao. M7 đứng đầu ở 99,30%, tiếp theo là M6 99,19%, M3 99,04% và M4 99,00%. M5 LeNet thấp nhất ở 98,50%. Khoảng cách giữa các model không lớn, cho thấy MNIST là bài toán tương đối dễ đối với CNN.

Một điểm cần chú ý khi đọc biểu đồ là trục y được giới hạn 95–100% trong notebook. Cách giới hạn trục giúp nhìn rõ chênh lệch nhỏ nhưng cũng làm chiều cao cột trở nên nhạy hơn. Vì vậy, số liệu cụ thể phải được đọc cùng bảng kết quả.

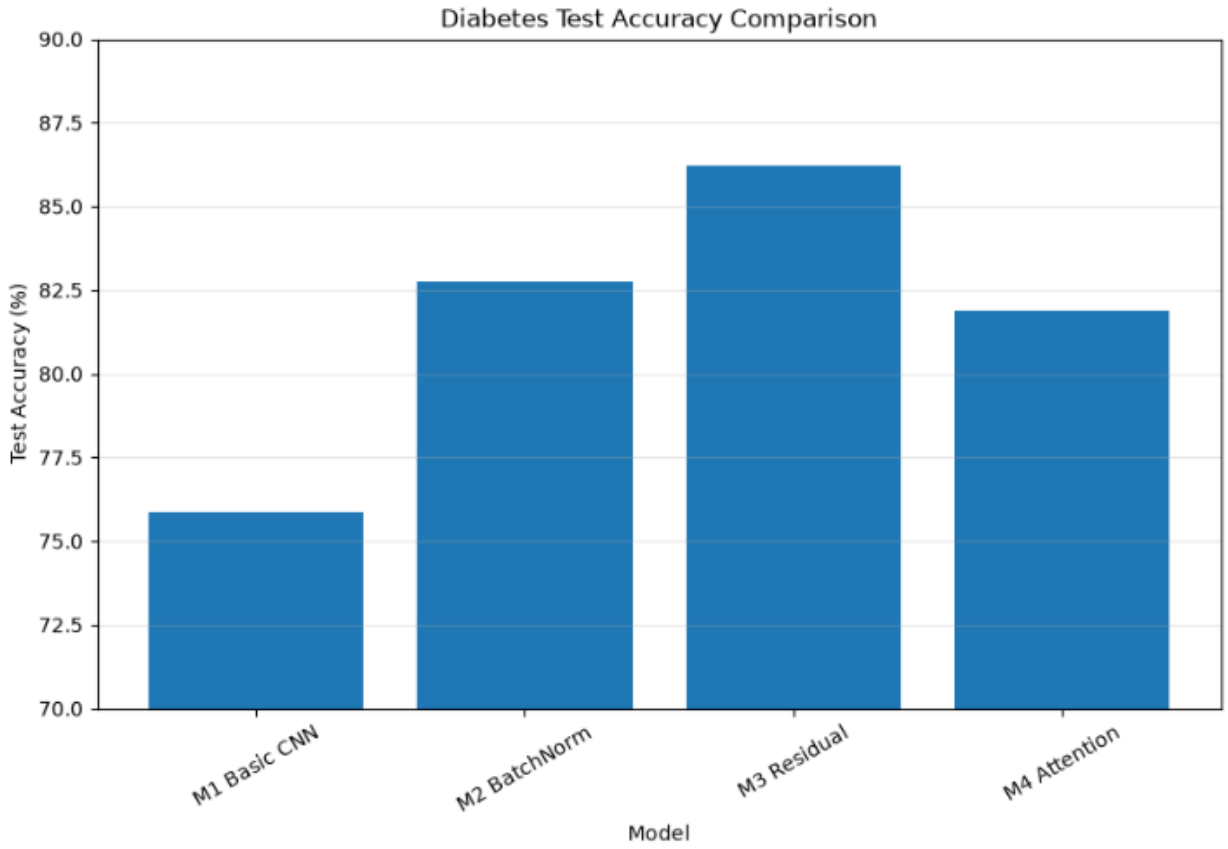
## 9.6. Test accuracy – Fashion-MNIST




*Hình 27. So sánh test accuracy trên Fashion-MNIST.*

M6 đạt 91,20%, M7 đạt 90,81%, M4 đạt 90,92% và M3 đạt 90,78%. M5 LeNet đạt 86,02%. So với MNIST, các cột có mức accuracy thấp hơn rõ rệt, thể hiện độ khó cao hơn.

## 9.7. Test accuracy – Diabetes

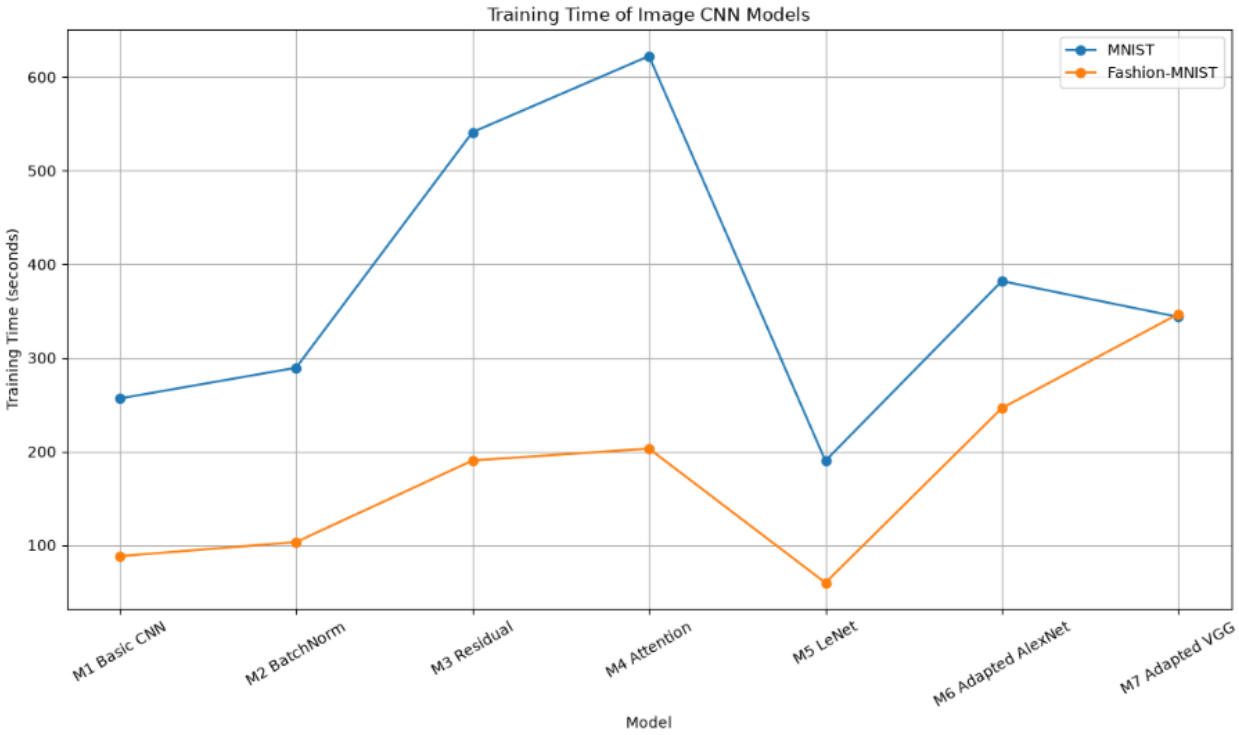



*Hình 28. So sánh test accuracy của bốn mô hình Diabetes.*

M3 Residual nổi bật với 86,21%, cao hơn M2 82,76%, M4 81,90% và M1 75,86%. Khoảng cách 10,34 điểm phần trăm giữa M3 và baseline là lớn nhất trong ba dataset.

Biểu đồ này là cơ sở trực quan cho nhận xét rằng residual connection có tác động tích cực trong thiết lập Diabetes cụ thể của notebook. Tuy nhiên, không nên tổng quát hóa rằng residual luôn tốt hơn attention trong mọi bài toán.

## 9.8. Training time

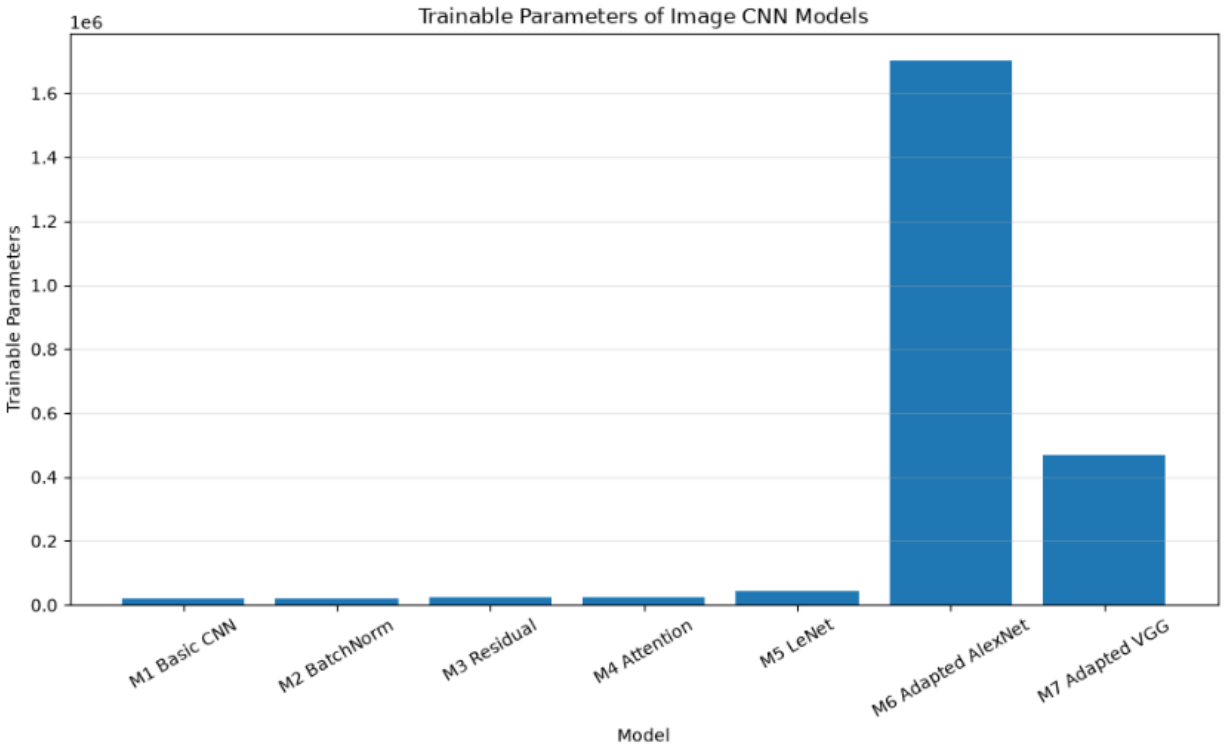



*Hình 29. Thời gian huấn luyện các mô hình ảnh trên MNIST và Fashion-MNIST.*

Training time trên MNIST cao hơn Fashion-MNIST trong nhiều trường hợp do cùng số epoch nhưng thời gian thực thi phụ thuộc vào nhiều yếu tố môi trường và batch processing. Trong nhóm MNIST, M4 Attention mất 622,51 giây, cao nhất; M5 LeNet mất 190,04 giây, thấp nhất.

Training time nên được dùng để thảo luận computational cost trong cùng môi trường chạy, không nên xem là một thuộc tính tuyệt đối của kiến trúc vì nó còn phụ thuộc phần cứng và implementation.

## 9.9. Trainable parameters

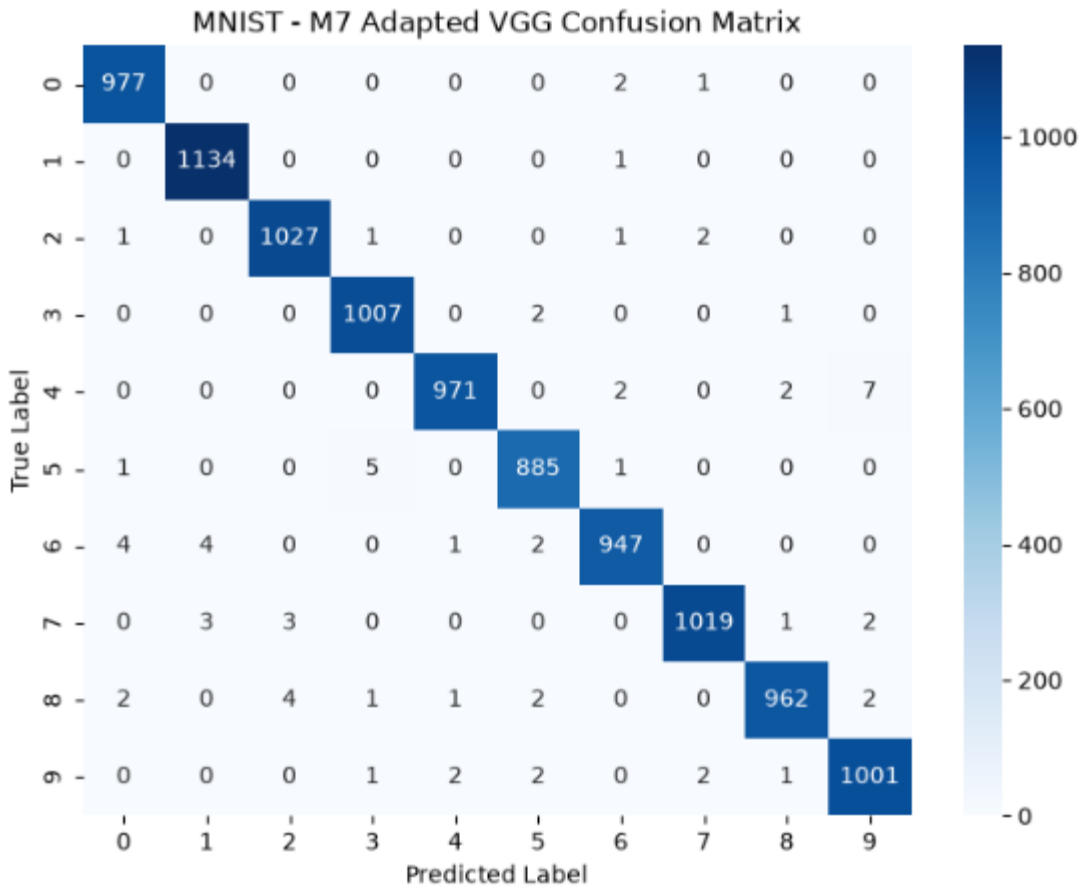



*Hình 30. So sánh số tham số trainable của bảy mô hình ảnh.*

Biểu đồ tham số thể hiện sự khác biệt rất lớn giữa các kiến trúc. M6 Adapted AlexNet có 1.701.130 tham số, cao hơn rất nhiều so với M1 20.490. M7 có 467.818 tham số, trong khi M3 và M4 chỉ khoảng 25 nghìn.

Sự khác biệt này giải thích một phần tại sao các classical architectures adapted theo phong cách AlexNet/VGG có computational cost cao hơn baseline. Tuy nhiên, M7 vẫn đạt accuracy MNIST cao nhất. Đây là ví dụ cho trade-off giữa complexity và predictive performance.

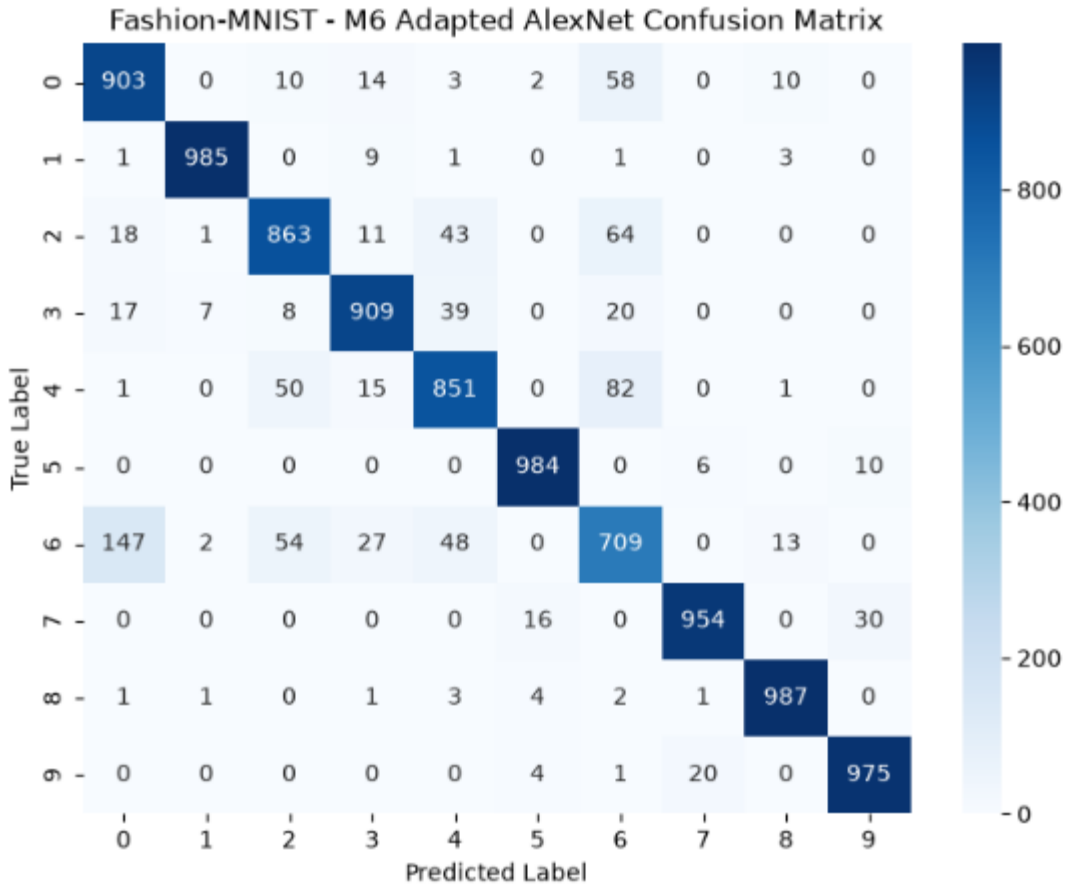
## 9.10. Confusion Matrix – MNIST




*Hình 31. Confusion Matrix của M7 Adapted VGG trên MNIST.*

Confusion matrix cho MNIST cho phép quan sát những cặp chữ số mà mô hình dễ nhầm. Vì accuracy tổng thể 99,30%, phần lớn khối lượng nằm trên đường chéo chính. Các ô ngoài đường chéo biểu diễn những trường hợp dự đoán sai.

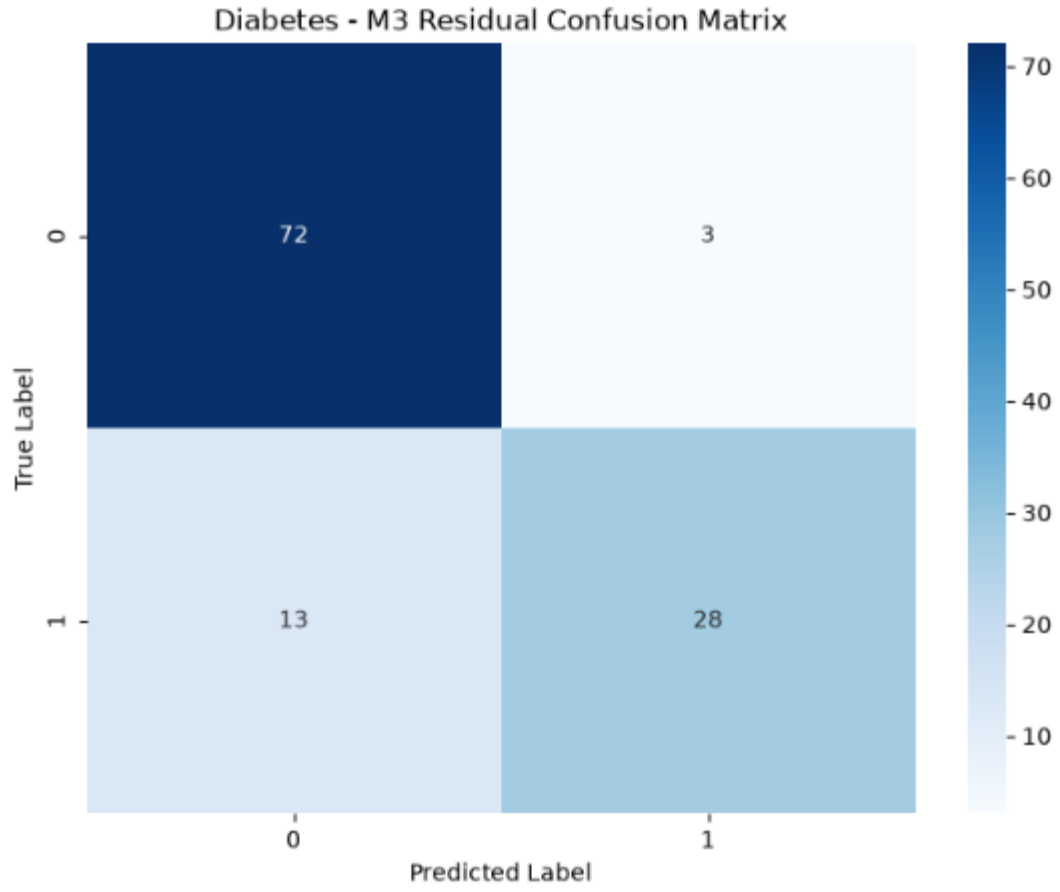
## 9.11. Confusion Matrix – Fashion-MNIST




*Hình 32. Confusion Matrix của M6 Adapted AlexNet trên Fashion-MNIST.*

Confusion matrix của Fashion-MNIST đặc biệt hữu ích vì các lớp thời trang có hình dạng tương tự nhau. Các lỗi ngoài đường chéo giúp xác định những cặp lớp mà mô hình khó phân biệt.

### 9.12. Confusion Matrix – Diabetes




*Hình 33. Confusion Matrix của M3 Residual trên Diabetes.*

Confusion matrix của Diabetes giúp kiểm tra trực tiếp số lượng mẫu Outcome 0 và Outcome 1 được phân loại đúng hoặc sai. Do đây là bài toán nhị phân, ma trận chỉ có kích thước 2×2 và có thể đọc trực tiếp theo TP, TN, FP và FN.

Đây là biểu đồ quan trọng để bổ sung cho accuracy 86,21%, vì accuracy không cho biết mô hình sai theo hướng nào.

## 10. SO SÁNH, THẢO LUẬN VÀ NHẬN XÉT

### 10.1. So sánh M1–M4 trên MNIST

| Model | Accuracy | Δ vs M1 | Parameters | Time (s) |
|---|---|---|---|---|
| M1 | 98.72% | 0.00 pp | 20,490 | 256.59 |
| M2 | 98.82% | +0.10 pp | 20,586 | 289.47 |
| M3 | 99.04% | +0.32 pp | 25,290 | 541.19 |
| M4 | 99.00% | +0.28 pp | 25,438 | 622.51 |

Từ M1 đến M4, accuracy tăng khi thêm BatchNorm và Residual, nhưng M4 không vượt M3. Chi phí thời gian tăng đáng kể, đặc biệt khi có residual block và attention.

### 10.2. So sánh M1–M4 trên Fashion-MNIST

| Model | Accuracy | Δ vs M1 | Parameters | Time (s) |
|---|---|---|---|---|
| M1 | 89.46% | 0.00 pp | 20,490 | 88.13 |
| M2 | 90.29% | +0.83 pp | 20,586 | 103.22 |
| M3 | 90.78% | +1.32 pp | 25,290 | 190.37 |
| M4 | 90.92% | +1.46 pp | 25,438 | 203.07 |

Fashion-MNIST cho thấy chuỗi cải tiến M1→M4 có xu hướng tăng accuracy liên tục. M4 cao nhất trong nhóm cải tiến với 90,92%, dù M6 vẫn đạt cao hơn toàn bộ nhóm ở 91,20%.

### 10.3. So sánh M1–M4 trên Diabetes

| Model | Accuracy | Δ vs M1 | Parameters | Time (s) |
|---|---|---|---|---|
| M1 | 75.86% | 0.00 pp | 1,761 | 0.75 |
| M2 | 82.76% | +6.90 pp | 1,857 | 0.95 |

| Model | Accuracy | Δ vs M1 | Parameters | Time (s) |
|---|---|---|---|---|
| M3 | 86.21% | +10.34 pp | 3,489 | 1.43 |
| M4 | 81.90% | +6.03 pp | 3,637 | 1.97 |

Diabetes thể hiện rõ nhất tác động của các cơ chế cải tiến trong thực nghiệm. M2 đã tăng 6,90 điểm phần trăm so với baseline, M3 tăng 10,34 điểm phần trăm. Tuy nhiên M4 giảm xuống 81,90%, cho thấy attention không phải lúc nào cũng cải thiện kết quả.

## 10.4. Classical CNN và model cải tiến

Trên MNIST, M5 LeNet đạt 98,50%, M6 Adapted AlexNet 99,19% và M7 Adapted VGG 99,30%. Trên Fashion-MNIST, các giá trị lần lượt là 86,02%, 91,20% và 90,81%. Vì vậy, Adapted VGG phù hợp nhất trong ba classical models trên MNIST, còn Adapted AlexNet có accuracy cao nhất trên Fashion-MNIST.

Kết quả cho thấy không thể dùng một bảng xếp hạng duy nhất cho mọi dataset. Đặc trưng dữ liệu và kiến trúc tương tác với nhau.

## 10.5. Accuracy và test loss không phải cùng một khái niệm

Accuracy chỉ quan tâm dự đoán cuối cùng đúng hay sai, trong khi CrossEntropyLoss còn phản ánh mức độ tự tin của dự đoán thông qua logits. Vì vậy, hai model có thể có accuracy gần nhau nhưng loss khác nhau.

Ví dụ Fashion-MNIST: M6 có accuracy 91,20% và test loss 0,242934; M7 có accuracy 90,81% nhưng test loss 0,242247. Sự khác biệt nhỏ này cho thấy model có loss thấp hơn không nhất thiết có accuracy cao hơn.

## 10.6. Accuracy, Precision, Recall và F1

Đối với image datasets, weighted Precision/Recall/F1 rất gần Accuracy vì mô hình có performance cao và các lớp được phân loại khá cân bằng. Đối với Diabetes, chênh lệch giữa các metric rõ hơn.

| Metric | Câu hỏi chính |
|---|---|
| Accuracy | Bao nhiêu mẫu được dự đoán đúng? |
| Precision | Trong dự đoán của một lớp, bao nhiêu là đúng? |
| Recall | Trong các mẫu thực sự thuộc lớp, bao nhiêu được tìm thấy? |
| F1-score | Cân bằng Precision và Recall như thế nào? |
| Confusion Matrix | Sai ở lớp nào và theo hướng nào? |

## 10.7. Complexity và hiệu quả

Một mô hình có accuracy cao nhưng quá lớn hoặc quá chậm có thể không phù hợp với môi trường hạn chế tài nguyên. Ngược lại, model nhẹ có thể chạy nhanh nhưng accuracy thấp hơn. Assignment 05 vì vậy có giá trị khi đánh giá cả parameter count và training time.

M6 Adapted AlexNet có 1.701.130 tham số, lớn nhất trong nhóm ảnh. M7 có 467.818 tham số nhưng đạt accuracy MNIST cao nhất. M1 chỉ có 20.490 tham số nhưng vẫn đạt 98,72% trên MNIST.

## 10.8. Vai trò của reproducibility

Notebook đặt seed 42 cho random, NumPy và PyTorch. Đây là bước cần thiết để giảm biến động do quá trình chia dữ liệu và khởi tạo model. Tuy nhiên, reproducibility tuyệt đối còn phụ thuộc phiên bản thư viện, phần cứng và các thao tác GPU.

## 10.9. Ý nghĩa học thuật của Assignment 05

Bài thực hành kết nối ba mức độ: công thức toán học, implementation và thực nghiệm. Người học không chỉ gọi một CNN có sẵn mà phải hiểu vì sao Conv2d, ReLU,

pooling, BatchNorm, residual và attention xuất hiện trong kiến trúc. Kết quả thực nghiệm sau đó cung cấp bằng chứng để kiểm tra các giả thuyết kiến trúc.

# 11. HẠN CHẾ CỦA THỰC NGHIỆM

## 11.1. Số epoch còn hạn chế

Image models chỉ được huấn luyện 5 epoch. Đây là thiết lập phù hợp để hoàn thành assignment trong thời gian hợp lý nhưng chưa đủ để khẳng định mỗi model đã đạt performance tối ưu. Một số mô hình có thể tiếp tục cải thiện nếu được huấn luyện lâu hơn.

## 11.2. Dataset Diabetes nhỏ

Diabetes chỉ có 768 mẫu, nhỏ hơn rất nhiều so với MNIST và Fashion-MNIST. Vì vậy, accuracy và F1 có thể nhạy với cách chia dữ liệu. Một đánh giá robust hơn có thể dùng repeated stratified cross-validation, nhưng notebook hiện tại sử dụng train/validation/test split cố định.

## 11.3. Conv1d trên dữ liệu bảng

Việc biểu diễn tám thuộc tính Diabetes thành chuỗi 1×8 là một cách tiếp cận thực nghiệm. Các feature không nhất thiết có quan hệ lân cận theo nghĩa không gian như pixel. Do đó, CNN Conv1d ở đây chủ yếu phục vụ mục tiêu nghiên cứu kiến trúc và không nên được xem là một kết luận rằng convolution luôn phù hợp hơn các mô hình tabular khác.

## 11.4. Classical architectures là phiên bản thích nghi

LeNet, AlexNet và VGG trong notebook là các phiên bản lightweight adapted cho 28×28 grayscale. Vì vậy, kết quả không thể dùng để so sánh trực tiếp với kết quả benchmark của kiến trúc nguyên bản trên ImageNet hoặc các dataset khác.

### 11.5. Training time phụ thuộc môi trường

Training time được ghi nhận bằng time.perf_counter. Đây là phép đo hữu ích trong cùng một notebook và cùng môi trường, nhưng không phải một đặc tính cố định của model. CPU/GPU, phiên bản PyTorch, batch size, data loading và tình trạng tài nguyên máy đều có thể làm thay đổi thời gian.

### 11.6. Không có tuning sâu

Learning rate, số epoch và kiến trúc được thiết lập trước. Notebook không thực hiện grid search hoặc Bayesian optimization. Do đó, mục tiêu chính là nghiên cứu và so sánh kiến trúc chứ không phải tìm hyperparameter tối ưu tuyệt đối.

### 11.7. Hướng cải thiện

- Huấn luyện nhiều epoch hơn và dùng early stopping.
- Dùng cross-validation cho Diabetes.
- Thử learning rate scheduler.
- Thử data augmentation cho MNIST/Fashion-MNIST.
- Phân tích confusion matrix theo từng cặp lớp khó.
- So sánh với các mô hình tabular như Logistic Regression, Random Forest hoặc Gradient Boosting.
- Đo thêm số FLOPs hoặc inference time nếu cần đánh giá triển khai.

## 12. KẾT LUẬN

Assignment 05 đã được triển khai theo một quy trình từ lý thuyết đến thực nghiệm. CNN được giải thích như một hàm hợp, bắt đầu từ neuron và phép biến đổi tuyến tính, sau đó thêm activation, convolution, pooling, flatten và classifier.

Phần kiến trúc nghiên cứu sự phát triển của CNN từ LeNet, AlexNet và VGG đến các ý tưởng như residual connection và attention. Trong thực nghiệm, bốn mô hình M1–M4 được xây dựng theo tiến trình có kiểm soát, còn M5–M7 đại diện cho ba kiến trúc kinh điển dưới dạng thích nghi.

Trên MNIST, M7 Adapted VGG đạt 99,30%. Trên Fashion-MNIST, M6 Adapted AlexNet đạt 91,20%. Trên Diabetes, M3 Residual đạt 86,21%. Các kết quả này cho thấy model tốt nhất phụ thuộc vào dataset và không có một kiến trúc duy nhất luôn đứng đầu.

Thực nghiệm cũng cho thấy việc đánh giá CNN cần nhìn đồng thời predictive performance và computational complexity. Accuracy, Precision, Recall, F1-score và Confusion Matrix cung cấp các góc nhìn khác nhau về chất lượng phân loại; parameter count và training time bổ sung góc nhìn về chi phí tính toán.

Quan trọng nhất, Assignment 05 cho thấy việc hiểu CNN không dừng ở việc gọi một lớp nn.Conv2d. Người học cần hiểu phép toán mà lớp thực hiện, cách feature map thay đổi kích thước, cách các cơ chế BatchNorm, residual và attention thay đổi luồng thông tin, và cách kiểm chứng những thay đổi đó bằng thực nghiệm.

# PHỤ LỤC: LINK DATASET VÀ SOURCE CODE

Dataset sử dụng trong Assignment 05:

- MNIST in CSV – Kaggle: https://www.kaggle.com/datasets/oddrationale/mnist-in-csv
- Fashion-MNIST – Kaggle: https://www.kaggle.com/datasets/zalando-research/fashionmnist
- Diabetes – Kaggle: https://www.kaggle.com/datasets/jamaltariqcheema/pima-indians-diabetes-dataset/data

Source code: https://github.com/tnd17/Assignment-05-PTHTTM-N05